

有界测角误差下干扰源的集合定位与覆盖约束路径规划

摘 要

针对半径 1800 m 圆域内数量、位置和接收半径未知的干扰源,本文以有界测角误差下的位置可行集为共同内核,将定位、搜索覆盖与清除路线联合规划。核心是分别证明“何处必能发现”和“何处必能清除”,再用同场景配对试验评价时间收益。

问题一:将示向度 $\pm 1^\circ$ 误差写为闭半平面交集,以边界交点枚举、凸包与衰退方向判别处理有界、无界、空集及退化区域,证明有界区域直径由顶点对实现。合法楔形反例的直径为 28.96681 m,以最远点对为直径的圆仍漏覆盖 0.26549 m;最小包围圆半径为 14.48690 m。因此定位直径不能替代清除点的覆盖半径认证。

问题二:利用首次接收事实得到接收半径下界 $R \geq \max(1000, \rho)$,将连续保证接收区化为三个径向端点及有限角极值判据,再按有限场景最坏定位直径优选第二测点。标准算例的 50 m 网格选点为(800,-600) m,评分 112.14914 m;局部 25 m 加密得到(850,-525) m、111.89311 m。输出连续接收区和 10%优选格点集,并明确数值优选的离散范围。

问题三:融合全向源的正负观测,采用 100 m 共享先导与六个外圈扫描点的弹性路径规划。圆周覆盖结合径向凸性保证发现完整性,19.9 m 包围圆认证及有限条带回退保证清除。100 场独立合成留出场景清除 1265/1265 个源,加权平均 233.723 s/源,较同场景基线降低 5.77%(配对重抽样 95%区间[4.67%,6.87%]);三次正式测试清除 13、14、11 个,平均 234.42—245.43 s/源。

问题四:针对未知发射方向的半平面可见性,构造原点、998 m 内环 7 点与 1848 m 外环 14 点的 22 点扫描布局,以局部近邻凸包包含条件保证任意方向可发现;2831 个三角片通过离线有理数复核。采用延迟调度、共享测向与矩形光学回退,回退最坏距离不超过 19.80 m。100 场合成留出清除 1321/1321 个源,平均 449.902 s/源,较 25 点基线降低 3.97%(95%区间[2.72%,5.14%]);官方演练 5 场清除 69/69 个,平均 438.87 s/源。三次正式测试清除 16、10、16 个,平均 336.07—592.76 s/源。

两问改进均以完整清除为约束,但压力试验保留了边缘场景退化,故不宣称逐场更优或全局时间最优。六份正式加密日志保持原名随支撑材料提交,合成试验、客户端记录与官方成绩分别报告。

关键词: 集合定位;连续覆盖;路径规划;示向度;干扰源清除

1 问题重述与分析

1.1 问题背景与任务

题目将干扰源限制在以原点为中心、半径为 1800 m 的圆域内，采用搭载测向机和光学清除设备的机器狗开展精准定位[1]。坐标单位为米，正东、正北分别对应横轴、纵轴正向。测向机输出从正东方向逆时针量取的示向度，误差位于 -1° 至 1° 之间；同一地点重复检测不会改变该误差。因此，仅沿单一观测方向移动或在同点重复检测，不能自动获得充分的二维定位信息。

问题一要求根据若干检测点及其示向度计算交会定位区域的直径，并判断以区域直径为直径的圆能否覆盖整个区域。问题二仅已知一个检测点的示向度，需要确定第二个检测点的选择策略及候选区域。前者回答“现有观测把源限制到多大范围”，后者回答“下一次观测在哪里获得更有价值的信息”。

问题三要求对数量未知的全向干扰源进行自动搜索、定位与清除，并通过演练和三次正式测试评价时间与清除效果。问题四进一步加入发射方向未知的定向干扰源，要求建立相应策略并完成同类测试。两问的模型、算法和结果分别设置于第 5、6 节。

1.2 前两问的建模思路

题面给出的是确定的误差区间，未指定可直接用于推断的误差概率分布。本文保留所有与观测相容的源位置，以集合交集表示信息积累过程。与直接将若干示向射线交点作为源位置相比，该表示能同时给出定位精度及保证范围，并可识别观测不足或互相矛盾的情形。

问题一的难点不是求出两条直线的交点，而是从全部误差边界恢复真实交会区域，并排除无界、空集等情形。对有界区域，凸性使连续的最大两点距离问题转化为有限顶点对问题；但最远顶点对只控制两点间距，不能替代对覆盖圆的检验。

问题二中，横向移动通常能改善交会角，却可能使机器狗离开有效接收范围。已有仅测角定位研究表明，观测者运动及测点几何会影响定位性能[2]。本文据此区分接收可行性与定位质量：前者用连续几何条件保证，后者在可行候选点上以有限场景计算比较，避免把缺失观测误判为良好定位。

本文四问共享同一集合定位内核：以位置可行集逐次求交保留观测信息，以最坏距离界保证发现与清除的完整性；问题一、二建立几何基础，问题三、四将该内核嵌入在线搜索与清除策略。总体建模与验证路线见图 1。

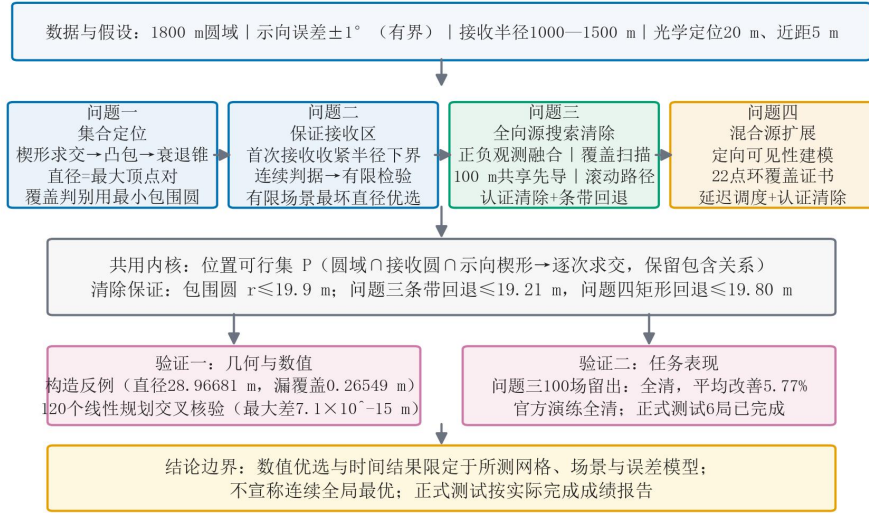


图1 本文四问的总体建模与验证路线

2 模型假设与符号说明

2.1 模型假设

假设一：同一定位过程中，干扰源位置、频道与有效接收半径保持不变，不同频道的信号互不影响。该假设直接采用题面对静态干扰源的描述，使跨检测点观测能够约束同一个位置变量。

假设二：每次有效示向度与真实方位角的圆周角差不超过 1° ，同点误差保持固定。计算将角度统一转换为弧度后使用三角函数，不额外假设误差独立、均匀或正态分布。若输入违反误差界，交会集可能为空，此时应报告不一致而非强行给出位置。

假设三：对全向源，在有效接收半径内能够接收信号，其半径位于 1000—1500 m；距离不超过 5 m 时可直接转入光学定位清除。问题二的保证接收分析仅适用于全向源，并以第一检测点确已获得示向度为前提。

假设四：仅在数值评价阶段离散第二点、源位置和第二次测角误差。该离散是求解近似，而非关于真实源分布的物理假设；连续保证接收区仍通过解析判据确定。圆域及接收圆的多边形近似均向外包络，不删除合法源位置。

2.2 符号说明

本文使用的主要符号列于表 1。位置和距离统一以米计，角度输入以度计；集合本身不赋予面积数值，只有计算其面积时才使用平方米。

表 1 主要符号与单位

符号	含义	单位
Ω	半径 1800 m 的源位置圆域	—
S_i, g	第 i 个检测点、未知源位置	m
θ_i, α	示向度、误差半界 ($\alpha=1^\circ$)	°
$u(t), e$	方向单位向量、局部纵轴单位向量	—
W_i, D	单次观测楔形、纯楔形交会区域	—
A, h	半平面约束矩阵、右端向量	—, m
$D_{\max}$	定位区域内最大两点距离	m
c, c^*, r^*	直径圆心、最小包围圆圆心、半径	m
R, ρ	有效接收半径、源到首次测点的距离	m
b, C	局部坐标下第二点、保证接收区	m, —
P_0, P_2	含先验的外包络、第二观测后外包络	—
$\hat{J}, \eta$	有限场景最坏直径、优选容忍度	m, —

3 问题一：有界误差下的交会定位与覆盖判别

3.1 将示向度转化为闭半平面

设第 i 次观测为检测点 S_i 及示向度 θ_i 。记二维叉积为 $\text{cross}(p, q) = p_x q_y - p_y q_x$ ，方向单位向量定义如下，其中 t 以度为输入。

$$u(t) = (\cos(\pi t/180), \sin(\pi t/180)) \quad (1)$$

合法源位置位于中心方向 θ_i 两侧各 α 的闭楔形内。采用叉积表达左右边界，可以直接处理跨越 0° 的观测方向，避免用普通实数区间比较角度造成错误。

$$W_i = \{p: \text{cross}(u(\theta_i - \alpha), p - S_i) \geq 0, \text{cross}(u(\theta_i + \alpha), p - S_i) \leq 0\} \quad (2)$$

$$D = \bigcap_{i=1}^k W_i = \{p: Ap \leq h\} \quad (3)$$

矩阵 A 包含 $2k$ 行，每行是单位法向量，向量 h 由边界线经过检测点的条件确定。本文在问题一中只计算题意所指的纯楔形交集 D ，不人为添加大矩形或目标圆域截断。否则，无界交集会被截成有限多边形，所求“直径”将依赖任意边框，而非观测信息。图 2 给出后文构造算例的检测点、观测边界和源位置。

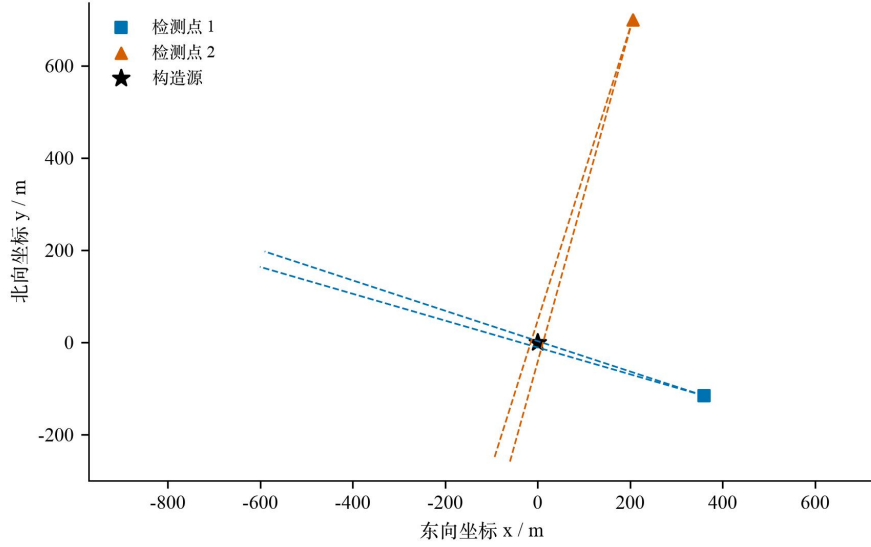


图 2 构造算例的两次示向观测及其误差边界

注：右下方检测点 1 为 S_1 ，右上方检测点 2 为 S_2 ；黑色星号为构造源，仅用于算例验证。

3.2 顶点生成、区域状态与算法步骤

每个非平行边界线对至多产生一个交点。枚举全部线对，将交点代入全部半平面约束，只保留满足约束的点，再去重并求凸包，即得到有界交会区域的顶点序列。该方法直接枚举实际边界，不依赖初始裁剪框。

设两条边界为 $A_1x+B_1y=C_1$ 、 $A_2x+B_2y=C_2$ ，其行列式与交点由下式计算。平行或数值上近似平行的线对不用于产生交点。

$$\Delta = A_1B_2 - A_2B_1, \quad x = \frac{C_1B_2 - C_2B_1}{\Delta}, \quad y = \frac{A_1C_2 - A_2C_1}{\Delta} \quad (4)$$

空集与无界性必须在计算直径前判断。当至少存在一个角宽小于 180° 的楔形时，交集不含完整直线；非空的此类二维多面体至少存在一个极点。因此，在精确算术下没有合格交点意味着区域为空。没有任何观测时则直接返回整个平面无界。

对于非空区域，定义衰退锥 K 。若存在非零方向 d 使所有法向投影不为正，则从任何可行点出发沿 d 走任意距离仍然可行，交集无界；反之，有界多面体的衰退锥仅含零向量。

$$K = \{d: Ad \leq 0\}, \quad D \text{ 有界} \Leftrightarrow K = \{0\} \quad (D \neq \emptyset) \quad (5)$$

在二维且已有观测楔形的条件下，若 K 含非零方向，则其边界射线与某条约束边界平行。因此可枚举每个单位法向量 (A_i, B_i) 对应的两个切向方向 $(B_i, -A_i)$ 、 $(-B_i, A_i)$ ，检查其是否满足 $Ad \leq 0$ 。这样既识别一般无界楔形，也保留单点、线段等低维有界交集。

计算流程见表 2。设最终顶点数为 n ，边界交点共 $O(k^2)$ 个，每个点检查 $O(k)$ 条约束，因此主要代价为 $O(k^3)$ ；凸包排序为 $O(k^2 \log k)$ ，顶点对直径枚举为 $O(n^2)$ 。整体可写为 $O(k^3 + n^2)$ ，适合检测次数较少且需要清晰边界判定的本题。

表 2 问题一算法流程与输出约定

步骤	计算内容	输出或处理
1	每条观测生成两个半平面	统一法向量与长度单位
2	枚举非平行边界交点并检查可行性	无观测：无界；无可行交点：空集
3	合格交点去重并求凸包	保留单点、线段与普通多边形
4	检查所有边界的正反切向方向	有非零衰退方向：无界、无有限直径
5	遍历凸包顶点对	输出最大距离及对应端点
6	检验覆盖并求最小包围圆	输出直径圆覆盖状态和安全覆盖圆

实现中，半平面包含容差为 10^{-7} m，边界平行判定阈值为 10^{-12} 。单位法向量使前者具有明确距离含义。数学算法的精确性不等于浮点数绝对精确；接近平行时交点位置对角度扰动敏感，应检查约束残差或提高数值精度，而不能将极大坐标直接视为高可信定位结果。

算法对区域状态的判别结果如图 3 所示：单次观测不足以约束二维位置时区域无界，合法的两次观测形成有界多边形，而互相矛盾的观测给出空集并触发不一致报告。

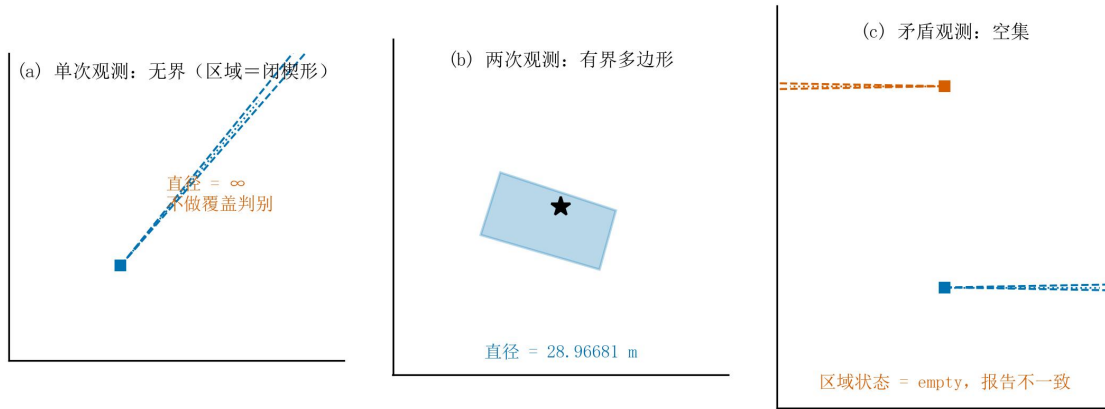


图 3 交会区域的三类状态：无界、有界与空集

注：观测为构造数据，仅用于状态判别演示；(b)中黑色星号为构造源。

3.3 定位直径的顶点对公式与证明

设非空有界交会区域的顶点为 v_1 至 v_n 。区域直径等于所有顶点对距离的最大值：

$$D_{\max} = \max_{p,q \in D} \|p - q\|_2 = \max_{1 \leq i,j \leq n} \|v_i - v_j\|_2 \quad (6)$$

证明：凸多边形内任意两点均可表示为顶点的凸组合，即 $p = \sum \lambda_i v_i$ 、 $q = \sum \mu_j v_j$ ，其中权重非负且分别求和为 1。于是利用三角不等式得到：

$$\|p - q\|_2 = \|\sum \lambda_i \mu_j (v_i - v_j)\|_2 \leq \sum \lambda_i \mu_j \|v_i - v_j\|_2 \leq \max_{i,j} \|v_i - v_j\|_2 \quad (7)$$

最大顶点对本身属于区域，所以上界可以达到，等式成立。该证明同时覆盖线段和单点：线段取两 endpoints，单点直径为 0。空集不赋予有限定位直径，无界区域的直径为无穷大；程序对这两种情形以区域状态和空值返回，避免与精确定位的零直径混淆。

3.4 直径圆覆盖的充要条件及反例

设 v_a, v_b 是一对实现直径的顶点，以 v_a, v_b 为直径的圆的圆心为 $c = (v_a + v_b)/2$ ，半径为 $D_{\max}/2$ 。对任意顶点 w ，有如下恒等式：

$$\|w - c\|_2^2 - \frac{\|v_a - v_b\|_2^2}{4} = (w - v_a) \cdot (w - v_b) \quad (8)$$

圆盘是凸集，因此它覆盖全部顶点当且仅当它覆盖整个多边形。结合上式，得到直径圆覆盖的充要条件：

$$D \subseteq B(c, D_{max}/2) \Leftrightarrow (w - v_a) \cdot (w - v_b) \leq 0 \text{ 对所有顶点 } w \text{ (9)}$$

直径定义只保证任意两点间距离不超过 D_{max} ，并不保证任意顶点到直径中点的距离不超过 $D_{max}/2$ 。例如等边三角形已经说明一般凸集不满足该性质。为确保反例符合本题 2°楔形交会结构，本文进一步采用表 3 中的两次实际构造观测；原始输入保留全部小数用于计算，表中仅为排版保留 8 位。构造源位于原点，两次角误差均在题设范围内。

表 3 产生非覆盖反例的构造观测

检测点	x / m	y / m	示向度 / °
S ₁	359.50558159	-115.32251209	162.77481542
S ₂	205.71977808	700.24209607	253.51621002

由两楔形求交得到表 4 所列四个顶点。枚举顶点对后，直径由 v_2 与 v_4 实现。图 4 同时绘出多边形、直径圆与最小包围圆，并以局部放大展示圆边界附近的差异。

表 4 交会多边形顶点及覆盖计算结果

对象	x / m	y / m	距离或半径 / m
v ₁	-16.69004445	-5.84824099	—
v ₂	8.11877713	-13.06769670	—
v ₃	11.53234759	-0.74522251	—
v ₄	-12.58164544	7.19480187	—
直径端点 v ₂ 、v ₄	—	—	28.96681451
直径圆心 c	-2.23143415	-2.93644742	14.48340725
最小包围圆心 c*	-2.45402117	-3.16384510	14.48690237

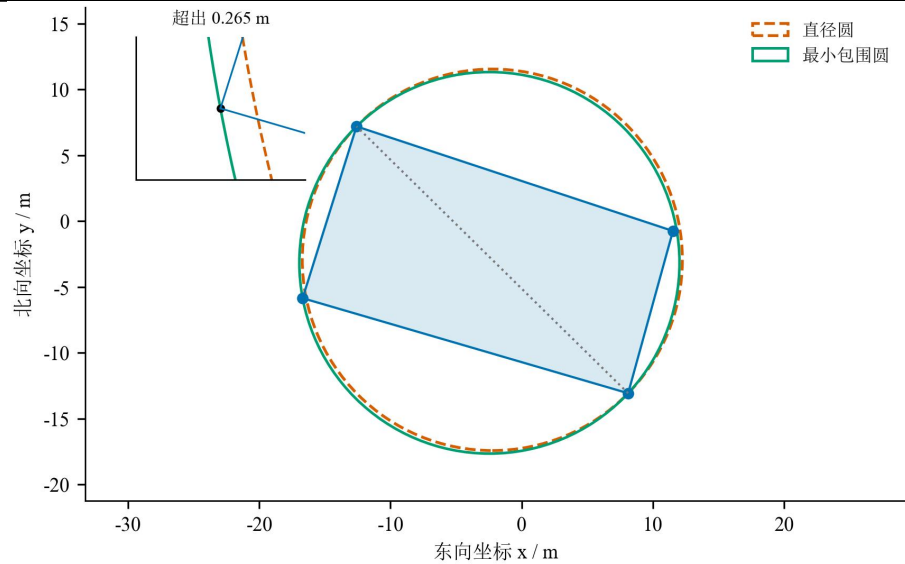


图 4 实际楔形交会反例中的直径圆与最小包围圆

计算所有顶点到直径圆心的距离后，最大超出量为 0.26548944 m，大于数值包含容差。故本例明确给出否定答案：以定位区域直径为直径的圆不能保证覆盖此定位区域。最小包围圆仅比直径圆半径增加约 0.00350 m，但圆心也发生移动；不能把两个半径之差误作原直径圆的最大漏覆盖量。

3.5 用最小包围圆连接定位与清除

若需要从区域信息选定一个能覆盖所有可能源位置的清除点，应求解最小包围圆。该圆心使最坏位置误差最小，其优化形式为：

$$r^* = \min_o \max_{1 \leq j \leq n} \|v_j - o\|_2, \quad c^* \in \operatorname{argmin}_o \max_j \|v_j - o\|_2 \quad (10)$$

平面有限点集的最小包围圆可由一个点、两个对径点或三个不共线边界点确定。若最优圆的边界点全部位于某个开半圆内，圆心可向这些点稍作移动而减小半径，与最优性矛盾；因此最优圆心属于边界点的凸包，二维情形可由至多三个边界点支撑。程序枚举单点零半径圆、两点直径圆及三点外接圆，逐一检查是否覆盖全部顶点并取最小者。

由凸性，只需覆盖顶点即可覆盖整个定位区域。于是，到 c^* 处清除的充分保证为 $r^* \leq 20 \text{ m}$ 。本例 $r^* = 14.48690 \text{ m}$ 满足该条件。一般而言， $D_{\max} \leq 40 \text{ m}$ 是存在 20 m 覆盖圆的必要条件，却不是充分条件；接近等边三角形的区域可能拥有较小直径而仍需大于 20 m 的覆盖半径。

3.6 约束递增试验与数值核验

增加一次合法观测相当于与新楔形求交，因此在非空有界条件下区域不会扩大，直径也不会增加。

$$D_{k+1} = D_k \cap W_{k+1} \subseteq D_k, \quad \operatorname{diam}(D_{k+1}) \leq \operatorname{diam}(D_k) \quad (11)$$

在上述两次构造观测基础上，继续添加不同检测点的合法观测，结果如图 5。检测点数从 2 增至 6 时，直径依次为 28.96681、23.75142、23.75142、23.69982 和 20.80626 m。第 4 次观测未改变直径，说明新增信息可能只切除非直径方向的局部区域；检测次数增加并不意味着每一步精度都严格提高。

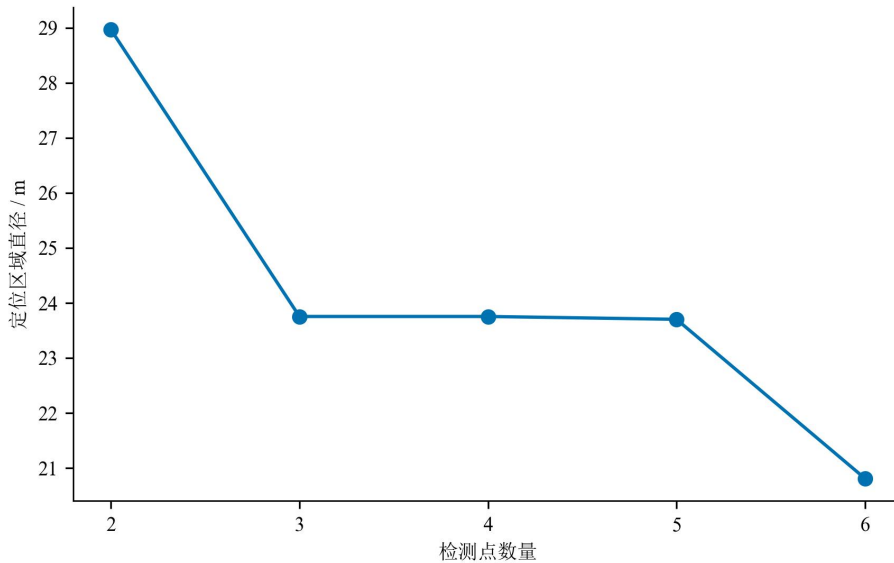


图 5 增加合法观测后定位区域直径的变化

另以独立线性规划计算 30 组测试区域在四个方向的支撑值，共 120 个线性规划，与顶点表示得到的支撑值比较，最大差异为 $7.11 \times 10^{-15} \text{ m}$ 。该结果核验所测样例中的约束求交实现，而算法对一般输入的依据仍是前述半平面、衰退锥和凸组合证明；有限测试不替代对所有边界情形的数学论证。

4 问题二：保证接收约束下的第二检测点优选

4.1 首次观测的可行源集合

取第一检测点 S_1 为局部原点，首次示向度 θ_1 方向为局部横轴，逆时针旋转 90° 为局部纵轴。令 $u=u(\theta_1)$ 、 $e=(-u_y, u_x)$ ，局部第二点 $b=(b_x, b_y)$ 转换为全局检测点的公式为：

$$S_2 = S_1 + b_x u + b_y e \quad (12)$$

由于已得到示向度而不是近距离反馈，真实源与 S_1 的距离大于 5 m ；由于已收到信号，源距不超过未知接收半径 R ，且 $R \leq 1500\text{ m}$ 。记源在局部坐标中为 $\rho u(t)$ ，其中 $|t| \leq \alpha$ 。结合目标圆域，得到真实源可行集 Γ ：

$$\Gamma = \{S_1 + \rho u(\theta_1 + t) : 5 < \rho \leq 1500, |t| \leq \alpha, S_1 + \rho u(\theta_1 + t) \in \Omega\} \quad (13)$$

标准算例取 $S_1=(0,0)$ 、 $\theta_1=0^\circ$ 。图 6 展示用于计算的 48 个有限源场景，包含 16 个径向位置与 3 个角向位置。它们用于评价第二点的表现，不是官方观测样本，也不代表真实源服从均匀分布。

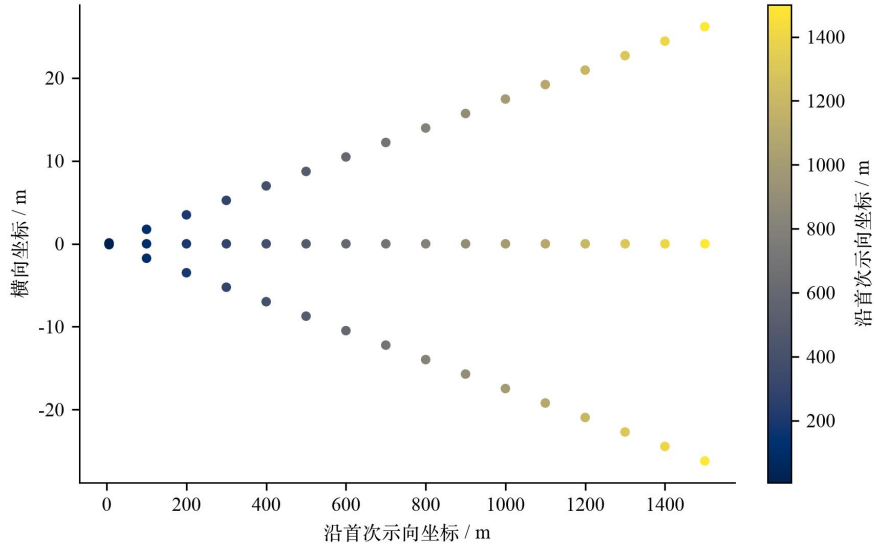


图 6 首次观测允许的狭长扇区及有限源位置场景

注：颜色按沿首次示向的轴向坐标编码；点位为计算场景，不表示真实源的概率分布。

4.2 连续保证接收区的定义

仅使用 $R \geq 1000\text{ m}$ 会忽略第一次已接收到信号所提供的信息：若源距 ρ 超过 1000 m ，该源的 R 必然至少等于 ρ 。因而在给定源位置时，可以使用更紧的下界 $R \geq \max(1000, \rho)$ 。第二点只要到每个允许源的距离都不超过这一最小可能半径，就能保证再次接收到信号。

$$R \geq \max(1000, \rho) \quad (14)$$

$$C = \{b : \|b - \rho u(t)\|_2 \leq \max(1000, \rho), \forall \rho \in [5, 1500], \forall t \in [-\alpha, \alpha]\} \quad (15)$$

这里将源距下端闭合到 5 m ，并暂不利用 Ω 对扇区的截断。闭合只加入极限位置，保证条件仍然有效；忽略圆域截断扩大了待防御的源集合，因而得到的是保守候选区域。每个源状态对应一个闭圆盘， C 是这些圆盘的交集，故为闭凸集，并且关于首次示向轴对称。

任意 $b \in C$ 对所有允许的全向源状态都有接收保证；若 $b \notin C$ ，只能说明这组保守条件未给出保证，不能断言某一实际源必然失去信号。实际源位置或目标圆域的额外信息仍可能使该点可用。这一区分避免将充分保证条件误写为对所有具体情形的必要条件。

4.3 无限约束的有限判据及证明

设 $m(b)$ 为第二点向量在所有允许源方向上的最小投影，则对固定 r ，最不利角度使该投影最小。

$$m(b) = \min_{|t| \leq \alpha} b \cdot u(t), \quad \max_{|t| \leq \alpha} \|b - \rho u(t)\|_2^2 = \|b\|_2^2 + \rho^2 - 2\rho m(b) \quad (16)$$

记 b 的极角为 β 。当 b 不为零时，投影等于 $\|b\| \cos(t-\beta)$ ；在短闭区间内，其最小值必位于两个角端点，或位于与 b 反向且落入该区间的角度。因此只需计算 $t=-\alpha$ 、 α ，以及落入区间的 $\beta+180^\circ$ 的等价角； $b=0$ 时投影恒为 0。角度是否属于区间按圆周归一化处理。

当 $5 \leq \rho \leq 1000$ 时，右侧接收半径为 1000，距离平方关于 ρ 为凸二次函数，其闭区间最大值在 $\rho=5$ 或 1000 处达到。当 $1000 \leq \rho \leq 1500$ 时，将接收半径平方 ρ^2 移到左侧，剩余式 $\|b\|^2 - 2\rho m(b)$ 关于 ρ 为仿射函数，只需检查 $\rho=1000$ 和 1500。两段合并后得到：

$$b \in C \Leftrightarrow \|b\|_2^2 + \rho^2 - 2\rho m(b) \leq \max(1000, \rho)^2, \quad \rho \in \{5, 1000, 1500\} \quad (17)$$

上述检验覆盖连续的全部 r 与 t ，而不依赖密集角度采样。由原点 $b=0$ 代入可知首次检测点位于 C 内；但原地重复检测的角误差不变，不会增加有用约束，因此“保证接收”并不等价于“值得检测”。候选点应继续经过定位质量筛选。

如图 7 对比本文的条件保证接收区 C 与仅使用 $R \geq 1000$ m 的保守区域，并叠加 20 m 检验格点；条件区更大，说明首次接收这一事实收紧了接收半径下界。用于定位评分的候选网格仍为第 4.5 节的 50 m 网格。

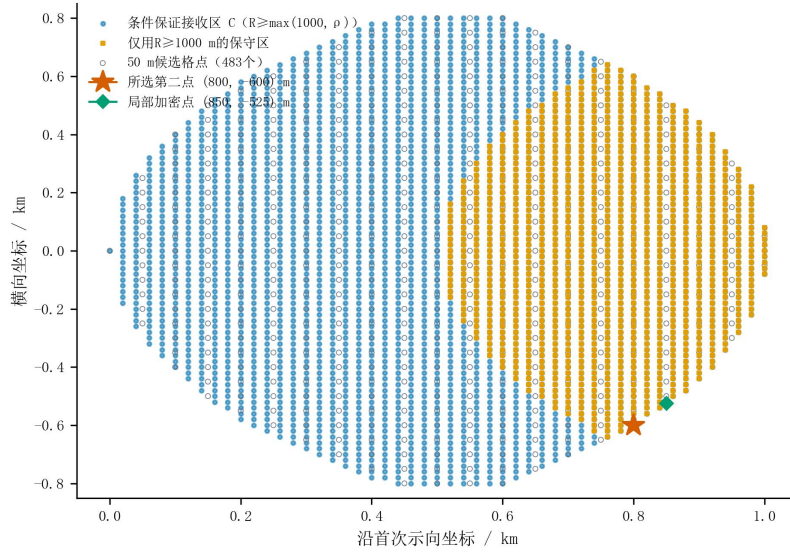


图 7 条件保证接收区与保守区域的对比

注：格点为 20 m 步长的数值检验； C 的边界仍以第 4.3 节连续判据为准。

4.4 以有限场景最坏直径评价定位效果

为评价第二点，首先构造包含全部合法源的先验外包络 P_0 ：目标圆域、首次检测点半径 1500 m 接收圆与第一观测楔形的交集。程序分别用外切正 128 边形表示两个圆。半径为 R_0 的圆，其外切多边形相对圆的最大径向外扩为：

$$\sigma_{128}(R_0) = R_0 \left(\sec \frac{\pi}{128} - 1 \right) \quad (18)$$

当 $R_0=1500$ m 与 1800 m 时，外扩分别约为 0.45191 m 和 0.54229 m。该近似方向使真源仍被包含。 P_0 没有挖去首次测点 5 m 内的圆孔，因此也是保守外包络。外扩量描述单个圆的几何逼近误差，不能直接作为最坏定位直径的总误差界。

$$P_0 = \Omega_{out} \cap B_{out}(S_1, 1500) \cap W_1 \quad (19)$$

对候选点 b 和有限源位置 g ，若第二点与 g 的距离大于 5 m，则第二次可能示向度由真实方位角叠加误差 ε_2 形成，得到相应楔形 W_2 及区域 P_2 。若距离不超过 5 m，则机器狗能够直接启动光学定位清除，该场景的残余定位不确定性记为 0。

$$P_2(b, g, \varepsilon_2) = P_0 \cap W_2(S_2, \text{bearing}(S_2, g) + \varepsilon_2) \quad (20)$$

$$\hat{J}(b) = \max_{g \in \Gamma_s, \varepsilon_2 \in E_s} \text{diam}(P_2(b, g, \varepsilon_2)) \quad (21)$$

Γ_s 与 E_s 分别为实际计算的有限源场景集及第二误差场景集，近距离场景按前述规则赋值。由于 P_0 为外包络，对每个固定场景计算得到的直径不会低估相应精确约束区域的直径；但对源与误差只抽取有限场景，可能漏掉更坏情形。因此 $\hat{J}$ 整体不是连续最坏直径的已证明上界，本文也不据此声称对所有连续源位置达到最优。

从几何上看，横向移动能够增大两条视线的交会角，减轻近共线导致的狭长交集。若暂以名义源位置进行小角度线性化，两条测角条带的交叠面积近似为：

$$A_{loc} \approx \frac{4\alpha_{rad}^2 \rho_1 \rho_2}{|\sin\varphi|}, \quad \alpha_{rad} = \frac{\pi\alpha}{180} \quad (22)$$

其中 ρ_1 、 ρ_2 为名义源到两检测点的距离， φ 为真实视线交会角。该近似只解释“避免近共线”的原因，既未考虑全部候选源，又在 φ 接近 0 或 180° 时失效，不能代替精确区域求交，也不能推出未知源情形下 90° 交会必为全局最优。

4.5 格点选择与候选优选区域

标准算例在局部坐标中设置横轴 0—1000 m、纵轴 -800—800 m、间距 50 m 的笛卡尔网格，用连续判据筛得 483 个保证接收格点。基础场景径向取 6 m 以及 100、200、...、1500 m，角向取 -1°、0°、1°，共 48 个源位置；第二次误差取 -1°、0°、1°，每个候选点最多评价 144 组组合。

$$C_{grid} = C \cap \{(50i, -800 + 50j) : i = 0, \dots, 20; j = 0, \dots, 32\} \quad (23)$$

$$b^* \in \operatorname{argmin}_{b \in C_{grid}} \hat{J}(b), \quad C_\eta = \{b \in C_{grid} : \hat{J}(b) \leq (1 + \eta)\hat{J}(b^*)\} \quad (24)$$

先按 $\hat{J}$ 由小到大优选；将评分四舍五入至小数点后 8 位后比较，相同者视为并列，再优先较短移动距离，最后按横纵坐标字典序确定输出。这样可以避免反射对称情形的浮点尾差影响选择。图 8 显示各保证接收格点的评分，图 9 显示 $\eta=0.10$ 时的优选格点。图中的离散点是本次数值候选集；连续区域 C 仍以第 4.3 节判据为准，不能由点间插值擅自补出新的优选保证。

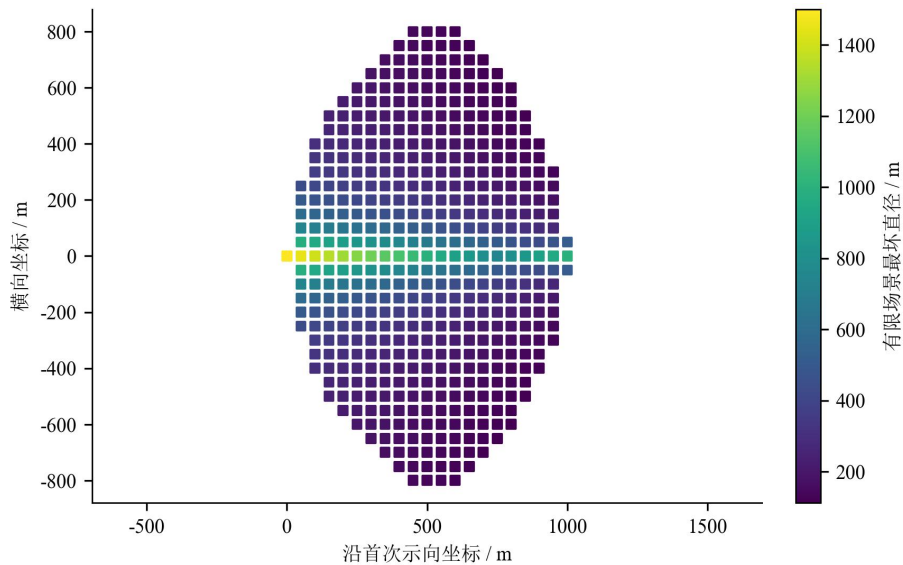


图 8 保证接收格点上的有限场景最坏定位直径

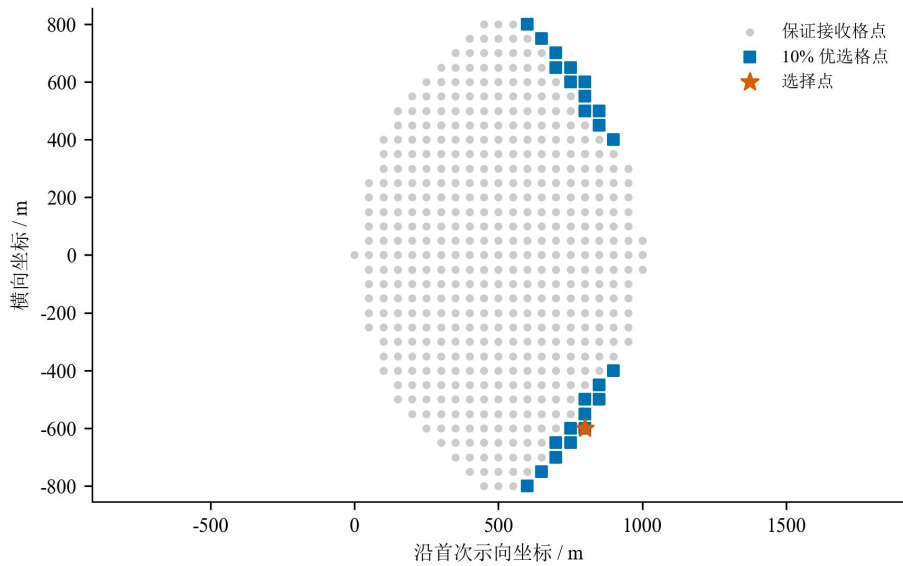


图9 保证接收格点、10%优选格点与所选第二点

基础网格优选点为 $b^*=(800, -600)$ m，移动距离为 1000 m， $\hat{j}=112.14914$ m。由于标准算例关于首次示向轴对称，其反射点(800,600) m 具有等价评分；输出负纵坐标来自并列处理规则，不代表南侧在一般情形下更优。 $\eta=0.10$ 时共有 24 个优选格点，它们集中在首次示向两侧具有较大横向分离的位置，而非紧贴示向轴。

4.6 典型选点对照与敏感性分析

以下表 5 对比沿示向前进、纯横向移动及斜向移动等典型选择。以(100,0) m 为例，虽能保证接收，但沿首次示向方向前进仍可能保留很长的径向不确定区，有限场景最坏直径达 1400.24481 m；可见移动后的接收安全性与几何辨识能力应分开评价。

表 5 标准算例不同第二点的有限场景比较

局部第二点 / m	保证接收	条件最坏直径 / m
(100, 0)	是	1400.24481
(0, 100)	否	770.07476
(500, 500)	是	173.16465
(750, 300)	是	176.80277
(800, -600)	是	112.14914

纯横向点(0,100) m 不属于保证接收区。表中 770.07476 m 是假设成功获得第二次示向度后计算的条件值，不代表该策略在所有允许源状态下可实现该定位效果。例如源位于首次示向轴上 1500 m 处且 $R=1500$ m 时，到该横向点的距离已超过 R 。因此它不能与具备连续接收保证的候选点作无条件优劣比较。

典型候选点的条件最坏定位直径对比如图 10 所示：沿示向前进虽保证接收，但径向不确定区很长；纯横向点定位几何较好，却不具备接收保证；斜向点位于两者之间。

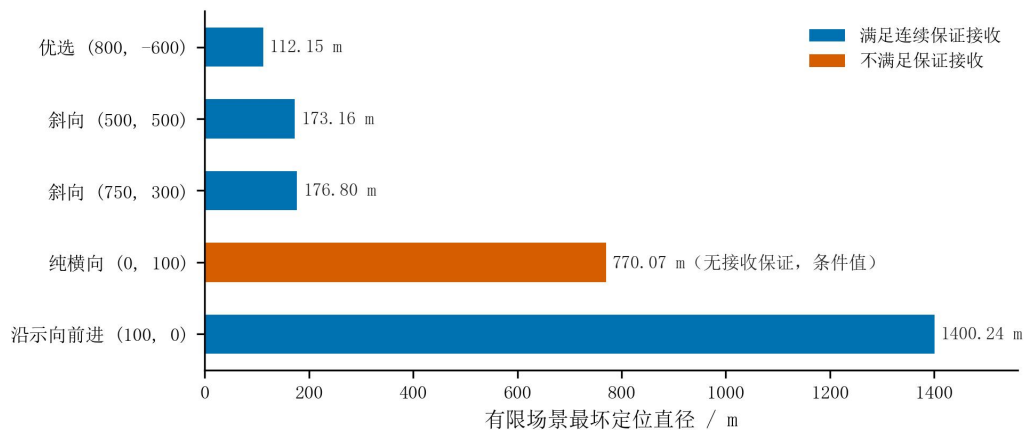


图 10 典型第二检测点的条件最坏定位直径对照

注: (0, 100) m 不满足连续保证接收, 其数值为条件值。

在粗网格优选点附近, 将横轴限制在 700—900 m、纵轴限制在-700—500 m, 网格间距改为 25 m; 经保证接收判据筛选后保留 40 点。同时将径向间隔改为 50 m 并保留 6 m 近端点, 角向及第二误差均增为 -1° 、 -0.5° 、 0° 、 0.5° 、 1° , 构成 155 个源位置与 5 个误差值。结果如表 6。

表 6 网格与场景加密结果

设置	基础计算	局部加密计算
候选点间距 / m	50	25
候选点数	483	40 (仅局部)
源位置数 $\times$ 误差数	48×3	155×5
所选局部坐标 / m	(800, -600)	(850, -525)
有限场景最坏直径 / m	112.14914075	111.89311189

加密后的优选点为(850, -525) m, 有限场景最坏直径为 111.89311 m, 比基础计算值小约 0.25603 m。两次计算同时改变了点网格与场景密度, 因此该差值只描述本次结果变化, 不能证明网格收敛, 也不是连续最优值的误差界。值得注意的是, 原点(800,-600) m 在加密场景中的评分仍为 112.14914 m, 说明本次局部比较未观察到该点的更坏离散场景。

优选容忍度 η 控制保留多少个近似优良的格点, 并不改变接收保证。表 7 给出敏感性结果: 从 5%放宽至 20%时, 优选格点数从 10 增至 58; 10%时保留 24 点, 可在定位精度接近的点中结合后续移动需求选择。若改变首次检测点、示向度或目标圆域相对位置, 应重新构造先验与场景并评分, 不应直接平移本算例的数值最优值。

表 7 优选容忍度对候选点数量的影响

容忍度 η	评分上限 / m	优选格点数
5%	117.75660	10
10%	123.36405	24
20%	134.57897	58

4.7 第二检测点策略总结

实际使用时, 先用首次检测点和示向度建立局部坐标与源可行域; 再根据第 4.3 节的连续判据剔除缺乏接收保证的格点; 随后计算每点的有限场景最坏直径, 选择评分较小且移动代价适当的点, 或输出 C_η 供后续任务选择。获得新示向度后, 将新楔形与已有定位集相交, 再按问题一的方法计算直径及覆盖圆。若近距离反馈表明源就在 5 m 内, 则直接进行光学定位清除。

这一策略给出了两层候选区域：C 提供连续状态下的接收保证，C_η提供指定网格与场景下的定位质量筛选。当前采用保守圆域处理、固定搜索窗口和有限误差场景；若追求连续极小极大最优，还需要对未采样源、角误差以及格点之间位置建立误差界或开展全局优化，现有结果不承担该结论。

5 问题三：全向干扰源的自动搜索、定位与清除

5.1 时间目标与在线信息状态

问题三的困难在于源数与位置均未知：机器狗既要缩小已发现源的定位区域，又要到尚未充分探测的位置搜索其他频道。若逐一完成“发现—定位—返回扫描点”，会反复经过相同区域；若只追逐已发现源，则可能遗漏远端目标。因此，本文将覆盖扫描点与已发现目标统一纳入滚动路径规划，用集合定位保持位置包含关系，并以连续覆盖条件决定搜索何时结束。

设实际源数为 N，成功清除的不同频道数为 N_c，累计移动距离为 L，检测次数为 M，实际切换频道次数为 K，失败清除次数为 F。根据题定速度及动作代价，总任务时间与两项评价指标写为式(25)、(26)。清除成功耗时 5 s，失败耗时 3 s；清除操作不改变测向机当前频道，因此 K 只统计实际检测动作引起的频道改变。

$$T = \frac{L}{5} + 5M + K + 5N_c + 3F \quad (25)$$

$$\zeta = \frac{N_c}{N}, \quad T = \frac{T}{N_c} \quad (26)$$

式(25)中各项均为秒，式(26)中平均时间只在 N_c>0 时定义。优化目标是在全部清除的前提下减少 T，而不是通过少清除难定位目标改善某一局部指标。线上程序不知道 N，不能用本地模拟器的真实源数停止；清到 10 个也不是完成条件。多个场景的总体平均时间按累计时间除以累计成功清除数计算，使源数较多的场景获得相应权重。

程序按频道维护三个互不相交的集合：尚未发现的频道 U、已发现但未清除的频道 H 以及已成功清除的频道 E。每个 H 中的频道保存首次测点、最近测点、示向度和可行多边形 P_c。观测为 direction 时建立或更新轨迹，为 near 时在产生该响应的位置直接清除；仅在 clear 返回 success 后将频道计入 E。无信号只更新该频道的位置约束，不直接将其判定为不存在。

理论测角误差仍为±1°。考虑示向度保留两位小数可能引入的舍入误差，在线计算采用α_c=1.005°作为包含数值裕量的建模半界；它不是新的误差概率分布。最小包围圆用于评价各 P_c 的最坏定位误差，最终算法参数汇总于表 8。

表 8 问题三最终策略的主要参数

参数	采用值	作用
实现测角半界α _c	1.005°	保留题定误差及舍入裕量
认证清除半径	19.9 m	相对 20 m 清除范围留裕量
非认证试清阈值	50 m，每频道至多 1 次	失败后继续观测
初始共享基线	100 m	24 个候选方向中选择
覆盖计算半径 R _c	999.99 m	小于最小接收半径 1000 m
外圈扫描点径向范围	801—1660 m	保持径向覆盖充分条件
弹性调整	每次规划至多 3 轮	无改进时提前结束

5.2 正负观测共同约束位置

每个新发现频道先按问题二建立圆域、首次 1500 m 接收圆及示向楔形的外包交集。后续有效示向度与现有多边形求交，从而持久化全部有用信息。初始圆的外切 128 边

形近似与前两问一致；相同频道和相同位置的检测结果使用缓存，避免为固定地点误差支付重复检测费用。

全向源在固定接收半径 R 内可见。设同一频道在 s 处有信号、在 n 处无信号，则源位置 g 满足式(27)。消去未知 R 并展开距离平方，可得到不含 R 的线性半平面约束式(28)。因此，“接收不到”与“接收到”的相对位置也能约束源，而不必等到所有观测都返回示向度。

$$|g - s|_2 \leq R < |g - n|_2, 1000 \leq R \leq 1500 \quad (27)$$

$$2(n - s) \cdot g \leq |n|_2^2 - |s|_2^2 \quad (28)$$

由 $R \geq 1000$ m，无信号测点 n 周围的 1000 m 圆内不可能存在该全向源。圆外截取可能产生非凸或不连通的区域，程序保留外部顶点和边—圆交点，再取凸包，作为剩余可行集的保守外包络；实现采用略小于 1000 m 的排除半径。凸化可能重新纳入部分已排除位置，因此会损失约束强度，但不会主动删除合法源位置。正负观测对的半平面随后继续裁剪该凸多边形。

这些约束依赖静态全向接收机制：同一频道的 R 在任务中不变，且接收范围内无漏检。定向源在半径内也可能无信号，因而本节的无信号排除规则不能直接用于问题四。网络异常或接口拒绝也不属于 no_signal；程序须将二者与合法的无信号响应区分。

5.3 由圆周覆盖推出整个目标圆域覆盖

搜索从原点开始，对所有尚未知的频道完成扫描。外圈初始设置 6 个检测点，分别位于半径 1200 m、相邻方位相差 60° 的位置。该初始布局提供覆盖基础，之后允许扫描点随访问路线移动，但每次移动都必须保留连续覆盖。某次只复测已发现频道的共享观测不算完整扫描；只有对所有仍未知频道都完成检测的位置，才能进入覆盖位置集合。

令外圈扫描点为 p_j ，其径向距离为 r_j 、极角为 β_j ，覆盖计算半径取 $R_c = 999.99$ m。在半径 1800 m 的圆周上，点 p_j 能够覆盖的方位半宽由余弦定理得到式(29)。计算中 β_j 与 ω_j 使用弧度；跨越 0 的区间拆为两段后，排序合并圆周区间。

$$\omega_j = \arccos \frac{1800^2 + r_j^2 - R_c^2}{3600r_j}, I_j = [\beta_j - \omega_j, \beta_j + \omega_j] \quad (29)$$

$$\cup_j I_j = [0, 2\pi] \quad (\text{按圆周区间合并}) \quad (30)$$

仅覆盖边界并不足以一般性地推出内部覆盖。这里还利用原点扫描与外点径向上界。固定任意圆周方向的单位向量 d ，按式(30)选一个覆盖 1800d 的外点 p ，令 $f(p) = |pd - p|^2$ 。由 $f(1800) \leq R_c^2$ 得式(31)；再代入 $p = 1000$ ，可得式(32)。

$$p \cdot d \geq \frac{1800^2 + |p|_2^2 - R_c^2}{3600} \quad (31)$$

$$f(1000) \leq \frac{4}{9} |p|_2^2 - 800000 + \frac{5}{9} R_c^2 \leq 980255.556 < R_c^2 \quad (32)$$

式(32)最后一步使用 $|p| \leq 1660$ m。函数 f 关于 p 为凸二次函数，故在闭区间 $1000 \leq p \leq 1800$ 上的最大值不超过两端值；同一个外点 p 覆盖该方向上的整段。半径不超过 1000 m 的内部圆盘由原点扫描覆盖。因此，在本节径向约束下，原点扫描加整周圆弧覆盖能够保证整个 1800 m 圆域至少落入一个已完成扫描的 1000 m 接收圆。

初始六点的 $r_j = 1200$ m，对应 ω_j 略大于 30° ，故相邻圆弧相接并有正裕量。每次弹性调整都以“已完成扫描的位置 + 全部尚待扫描的位置”重新检查式(30)，只接受保持整周覆盖的移动。全部外圈扫描任务执行完毕后，再用实际扫描位置复核同一条件，避免把规划位置当作已经获取的观测。旧的 100 m 网格剩余格数仅保留为诊断量，不作为该最终版本的结束判据。

外圈六个扫描点的整周覆盖与径向覆盖条件如图 11 所示。

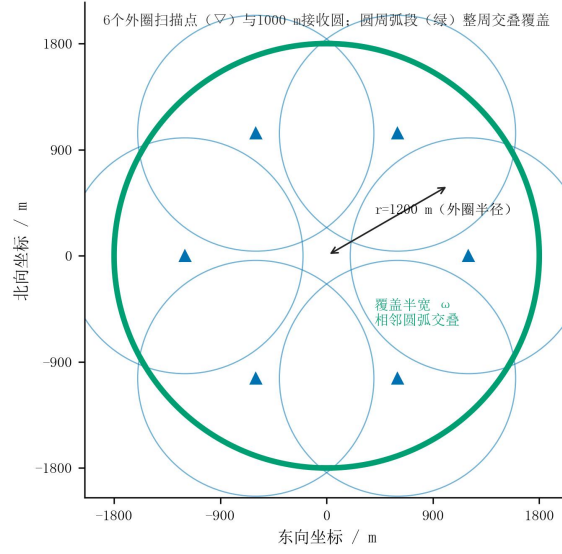


图 11 外圈六点的整周覆盖与径向覆盖条件

注：覆盖计算半径 $R_c=999.99\text{ m}$ ；绿色弧段为各扫描点的覆盖范围。

若所有扫描任务已执行、覆盖条件成立且 H 为空，则未发现频道在整个目标域内都不存在未清源，任务结束。若已成功清除 16 个不同频道，则由题定总数上界也可立即结束。上述完整性结论以合法响应、题定模型及充分运行预算为前提；若预算耗尽、接口异常或认证清除失败，程序应报告对应状态，不能将中断记为完整清除。

5.4 初始先导观测与停点共享测向

原点扫描可能同时发现多个源，但各自可行域沿初始示向方向较长。直接按其包围圆心规划路线，会因距离估计不准产生绕行。最终策略在极点扫描后仍有至少两条未清轨迹时，增加一次 100 m 的共享先导移动：从 0° 至 345° 、每隔 15° 产生一个候选方向，并在同一先导点复测所有已知未清频道。

设原点为 O ，某轨迹的包围圆心和半径为 c_c 、 r_c ，候选先导点为 q 。以 O 与 q 观察 c_c 的名义交会角记为 $\varphi_c(q)$ ，程序用式(33)比较 24 个候选位置。0.035 近似表示两倍角误差的弧度量级，0.015 为防止近共线分母过小的截断值。该代价是用来选方向的启发式估计，不是对真实定位半径的严格上界。

$$\Psi(q) = \sum_{c \in H} \min(r_c, \frac{0.035/|q - c_c|/2}{\max(|\sin \varphi_c(q)|, 0.015)}) \quad (33)$$

先导长度采用开发集比较选定的 100 m。该点只测已知频道，不计入完整覆盖扫描集合；若没有足够的已知轨迹，则不支付这次先导移动。与此同时，机器狗在定位、清除或扫描停点，针对其他有潜在收益的已知频道共享测向，使一次移动同时改善多个 P_c 。

共享触发条件为：包围圆半径仍大于 19.9 m，当前位置到名义圆心不超过 1150 m，与该频道上次测点相距至少 60 m，并且两条名义视线交会角的正弦绝对值大于 0.12，或当前位置到名义圆心小于 150 m。1150 m 仅用于判断是否值得尝试，并不构成保证接收半径；实际 no_signal 仍按第 5.2 节更新信息。检测次序优先当前频道，以降低切换费用。

5.5 覆盖点与清除目标的联合路径规划

在每次完成一项扫描或定位任务后，将全部待扫描点与已发现目标的包围圆心共同组成任务位置集合。以当前位置为起点，用最近邻构造开放路径，再用 2-opt 反转局部次序以减少长度。任务不要求回到原点，因此不添加末点返回原点的边。式(34)给出

该次规划比较的路径长度，其中 z_0 为当前位置， z_1 至 z_m 为按访问次序排列的任务位置。

$$L_{plan} = \sum_{i=1}^m |z_i - z_{i-1}| / 2 \quad (34)$$

路径中目标位置只是当前可行集的名义中心，并非已知真值。程序只执行规划中的第一项任务，取得新观测后更新可行集并重新规划，因此不会将一条初始估计路线僵硬执行到底。2-opt 最多尝试 80 轮局部改进，若没有可降低长度的反转则提前结束；该过程仍是启发式搜索，没有全局最短路保证。

在开放路径基础上，继续调整尚未访问的扫描点位置。对一个扫描点 p ，以路径中相邻位置 a 、 b 定义局部代价 $\ell(p) = |a-p| + |p-b|$ ；若为末点则只计前一段。候选包括朝线段 ab 投影方向的分级移动，以及步长 100 m、30 m 的八方向移动。只有同时缩短局部路径、满足 $801 \leq |p| \leq 1660$ m 并保持式(30)的候选才被接受。每次规划至多做三轮弹性调整，某轮没有改进即提前结束。

$$\min_p \ell(p), \quad 801 \leq |p| \leq 1660, \quad \cup_j I_j = [0, 2\pi] \quad (35)$$

这一步利用了覆盖圆弧之间的重叠余量：扫描点并不必永远固定在规则六边形上，而可以向当前任务路线靠近。最终版本保留六个外圈扫描任务，不采用早期原型中的任意增扫与删点逻辑。这样既减少行进距离，又使覆盖证明与实际终止条件保持一致。图 12 给出开发种子 1 的源位置，仅用于展示场景；图 13 比较同一场景中上一版与最终策略的完整动作路线。

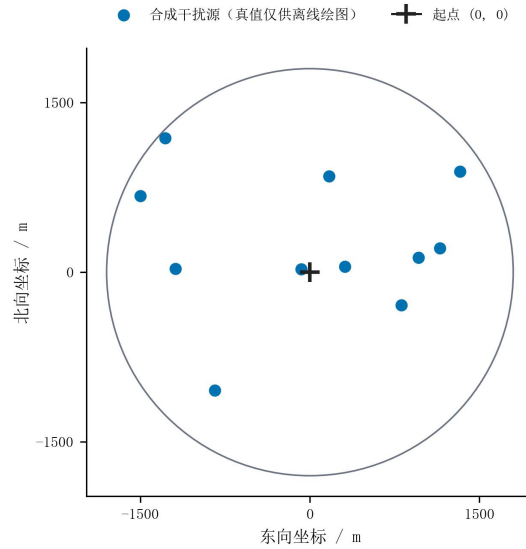


图 12 开发场景示例：种子 1 的源位置与起点

注：源真值仅供离线绘图和核验，在线策略不可读取。

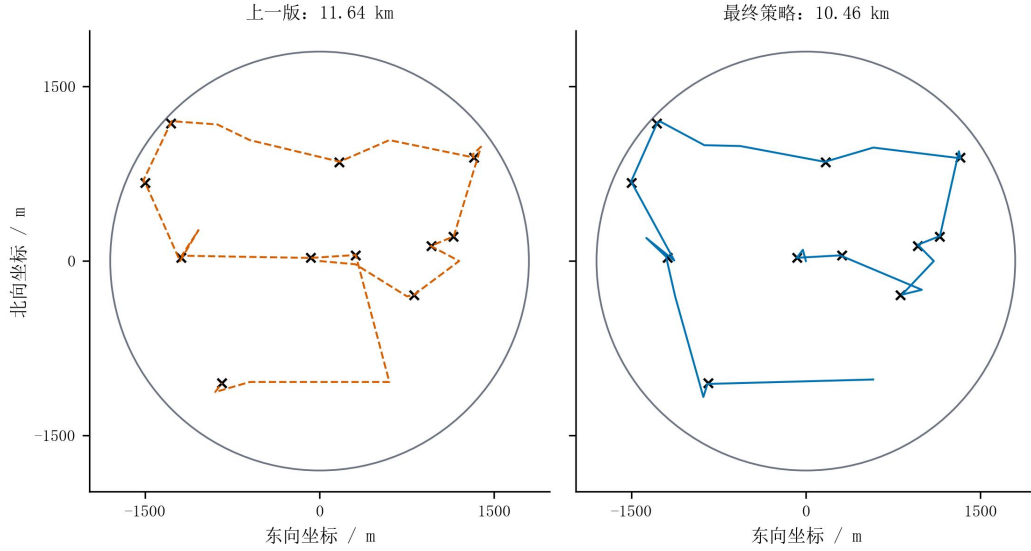


图 13 同一开发场景的动作路线对比

注：上一版用橙色虚线表示，最终策略用蓝色实线表示；该图为开发种子 1，不作为独立留出结果。

5.6 认证清除、有限试探与确定性回退

对于当前选中的频道，首先计算 P_c 的最小包围圆。若 $r_c \leq 19.9$ m，则移动至圆心实施清除。只要真实源仍在 P_c 内，圆心到源的距离就小于 20 m，因而具备几何保证。若 $19.9 < r_c \leq 50$ m 且该频道尚未进行过非认证尝试，则允许先到圆心试清一次；50 m 是尝试阈值，不意味着光学设备的清除半径扩大。

$$r_c \leq 19.9 \Rightarrow |g - c_c|/2 < 20 \quad (36)$$

非认证尝试返回 success 才记入成功数；若失败，立即在该圆心位置测向，用已经支付的移动换取新信息，再继续缩小区域。每频道最多一次非认证尝试，防止对同一不确定中心反复付费。未满足试清条件时，采用名义中心附近的横向观测点；单条未充分约束的示向采用前进 300 m、侧移 ± 75 m 的候选，避免一开始支付过长的横向行程。由正负观测已将区域明显缩小时，不再机械使用这一单示向规则。

局部交会在有限步内未完成时，回到首次测点与首次示向构成的确定性条带。首次合法示向意味着源在局部横坐标 0—1500 m、纵坐标 $\pm w$ 之间，其中 $w = 1500 \sin(\alpha_c)$ 。在纵坐标 $\pm w/2$ 的两条线上，沿横轴以不超过 28 m 间隔放置清除点，共 110 点并按蛇形顺序尝试。任意条带内位置到最近清除点的距离满足式(37)，故预算充足时能通过有限扫描完成清除。

$$d_{strip} \leq \sqrt{14^2 + (w/2)^2} < 20, \quad w = 1500 \sin(\pi \alpha_c / 180) \quad (37)$$

回退清除不依赖再次收到无线电信号，因而不会把一次 no_signal 演变为放弃目标。开发、压力与新留出集合中最终策略均未触发条带回退，但这只说明所测场景中的加速有效，不能据此删除完整性保障。另需区分认证尝试失败与普通试探失败：前者意味着模型、数据或接口不一致，应报告异常，不能静默忽略。

5.7 算法迭代与本地配对验证

本地实验采用同一场景配对：源位置在圆域内按面积均匀生成，频道互异，接收半径在 1000—1500 m 内生成，源数在 10—16 之间。误差由“种子、频道、测点坐标”确定的固定空间散列生成，再按两位小数返回示向度，使不同策略在同一测点获得相同误差，而不会因动作顺序改变而取得不同随机序列。这些分布仅用于数值试验，并非题目规定的真实分布。

迭代中并非所有机制都会改善性能。例如，早期将大量沿途停点转为全频道扫描，虽然缩短了部分移动，却增加检测和切换负担，在同一 10 场筛选集上由 251.254 上升

至 281.944 s/源。因此最终选择保留覆盖结构，重点压缩已确定扫描任务与目标任务之间的行程。表 9 列出若干关键比较；不同场景数量的行分别属于不同批次，不能直接用其数值计算跨批次改善率。

表 9 关键迭代的同批次比较（本地合成场景）

批次 / 场景数	策略	平均时间 / (s/源)
初筛 / 10	上一版共享定位与固定覆盖	251.254
初筛 / 10	增扫型联合搜索原型	281.944
初筛 / 10	上一版加入可见性约束	246.975
组合比较 / 40	上一版	246.865
组合比较 / 40	弹性覆盖+100 m 共享先导	229.368
组合比较 / 40	弹性覆盖+200 m 共享先导	230.023

最终入口采用 100 m 先导、50 m 一次试清和至多三轮弹性调整。参数选择仅使用开发种子 1—40，随后在压力场景及新种子 20001—20100 上评价，不依据新留出结果反向调整这组参数。表 10 同时报告清除率与耗时：开发 40 场、压力 50 场和新留出 100 场，最终策略分别清除 526、640、1265 个源，全部达到 100%。上一版在相同场景也全部清除，故时间差异不是通过漏清换取。

表 10 最终策略与上一版的配对验证结果

集合	场景 / 源数	上一版 s/源	最终 s/源	时间降低
开发集	40 / 526	246.865	229.368	7.09%
压力集	50 / 640	254.427	244.910	3.74%
新留出集	100 / 1265	248.044	233.723	5.77%
三集合合计	190 / 2431	249.469	235.726	5.51%

跨场景的相对改善按式(38)计算，其中 $T_{b,i}$ 与 $T_{u,i}$ 分别表示同一场景中上一版和最终策略的总时间。新留出集 100 场中有 84 场最终策略更快、16 场更慢。对场景对整体重抽样 4000 次，随机种子为 20260912，得到改善率的 95%百分位区间为 [4.67%,6.87%]。该区间描述合成生成器内的抽样不确定性，不是所有合法布局的性能保证，也不作为官方成绩区间。

$$\Delta_T = 1 - \frac{\sum_i T_{u,i}}{\sum_i T_{b,i}} \tag{38}$$

如图 14 进一步给出同一开发场景中两种策略的清除进度：最终策略在相同累计清除数下用时更短，且两者均在任务末段保留了一段扫空时间。

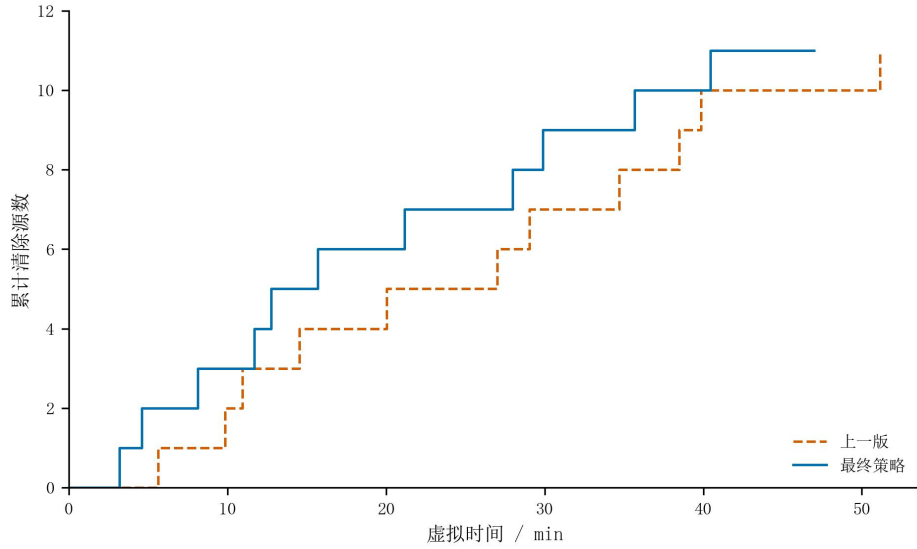


图 14 同一开发场景的清除进度对比

注：数据为开发种子 1，不作为独立留出结果。

如图 15 所示，逐场比较新留出集平均时间；对角线下方代表最终策略用时更少。最差退化出现在种子 20009：上一版 277.594 s/源，最终策略 307.979 s/源，增加 30.385 s/源。可见最终策略改善整体平均表现，但没有逐场支配上一版。新留出集中 18 场不超过 200 s/源，整体均值仍为 233.723 s/源，不能将部分优秀场景称为总体已达到 200 s/源。

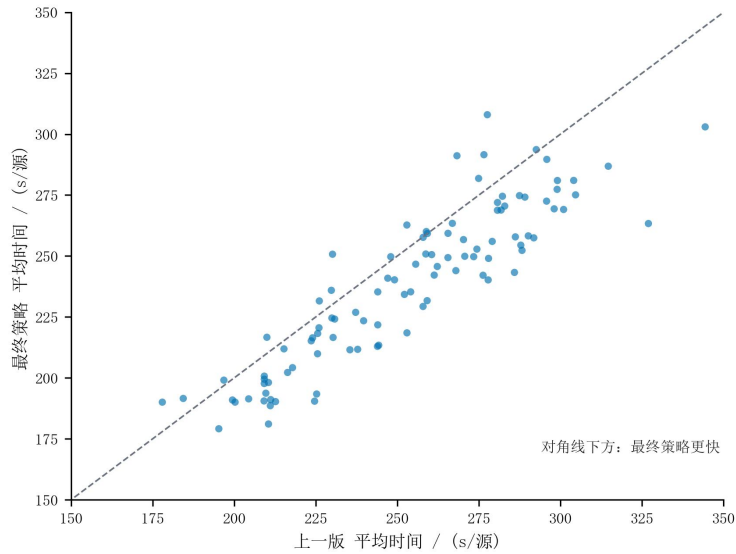


图 15 新留出集 100 场的逐场配对耗时

注：每点对应同一场景的两个策略结果；虚线表示两者相等。

如图 16 所示，将新留出集的时间按动作拆分。最终策略移动、检测、切换、清除分别为 175.006、44.996、8.557、5.164 s/源，合计 233.723 s/源；上一版分别为 189.033、45.431、8.580、5.000 s/源。主要收益来自移动时间降低约 14.026 s/源，约对应每源少走 70.13 m；试清增加约 0.164 s/源，但整体移动节省更大。移动仍占最终总时间约 74.88%，因此后续改进应继续关注观测收益与路线长度的协同，而非仅减少计算运行时间。

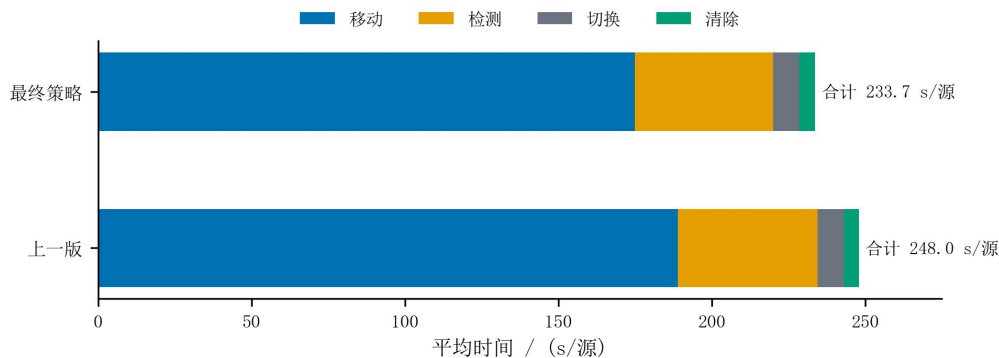


图 16 新留出集的平均动作耗时分解

注：依次统计移动、检测、切换与清除；失败试清耗时计入清除。

为检查结构性弱点，设置五类压力场景，每类 10 场、128 个源。边缘分布将源置于 1799.9 m 圆周且 $R=1000$ m；聚簇分布将源集中于外环小簇；最小接收半径组令全部 $R=1000$ m；固定偏差组按频道固定 $\pm 1^\circ$ 误差；端点误差组令各测点误差取 $\pm 1^\circ$ 。表 11 及图 17 给出全部类别，保留表现退化的边缘组。

表 11 五类压力场景的分组结果

场景类型	上一版 s/源	最终 s/源	清除比例
固定偏差 (bias)	244.862	236.260	128/128
外环聚簇 (cluster)	169.407	159.396	128/128
圆周边缘 (edge)	345.441	346.790	128/128
端点误差 (extreme)	249.575	233.949	128/128
最小半径 (minrange)	262.851	248.154	128/128

最终策略在四类压力分组中更快，但圆周边缘组由 345.441 增加至 346.790 s/源，约退化 0.39%。这一结果说明，当目标位于探测范围边缘且覆盖余量较小，共享先导和路径局部调整的附加代价可能无法由绕行减少抵消。图 17 展示各组逐场分布，避免只用分组均值掩盖困难场景。

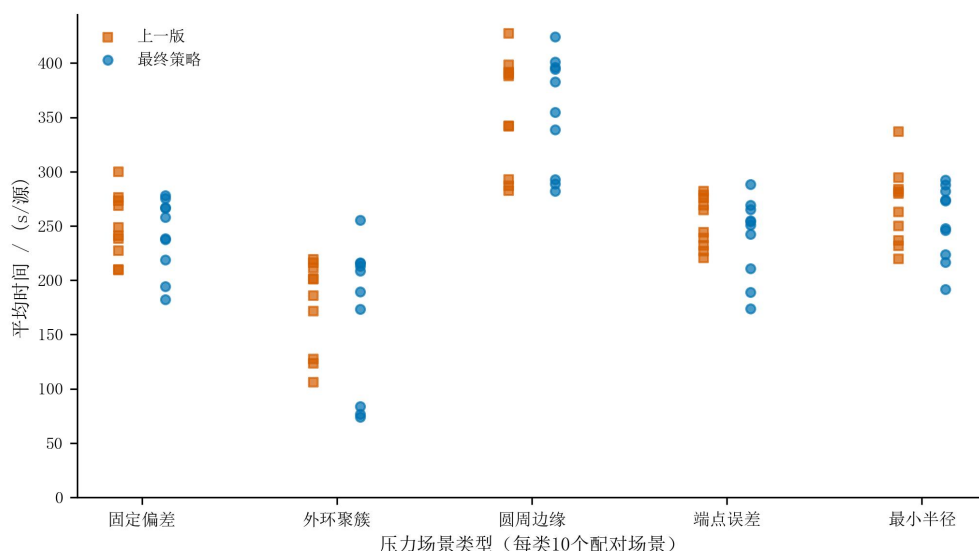


图 17 五类压力场景的逐场平均定位清除时间

注：每类为 10 个配对场景；方形为上一版，圆形为最终策略。

还可用真值构造离线理想信息下的保守下界，说明目前尚未证明时间最优。将原点与 N 个真实源中心的最小生成树长度记为 L_MST ；若一条清除路径长为 L ，把其访

问的清除位置替换为相应源中心，每条边最多增加 40 m，总增长不超过 40N。由最小生成树不长于该连接路径，得到式(39)。真值只用于离线诊断，不能作为在线控制输入。

$$\frac{T}{N} \geq \frac{\max(0, L_{MST} - 40N)}{5N} + 5 \quad (39)$$

该下界忽略检测、切换与未知源搜索的代价，通常较松，不能以当前算法接近某个经验平均值就判定全局最优。本地程序运行时间也不同于题定任务虚拟时间：新留出集中最终策略实测最大运行时间约 6.615 s，受机器与负载影响，仅用于评估该次实现的计算开销。

5.8 客户端运行记录与正式测试数据边界

除本地配对试验外，项目保留了最终策略在 2026 年 9 月 12 日 15:05—15:07 的五场客户端运行记录。表 12 列出全部五场，不将其中最好的一场代表总体表现。客户端共记录 63 个成功清除频道、总虚拟时间 14032.233409 s，加权平均为 222.733864 s/成功清除源；简单平均五个场均时间会得到不同数值，因此本文采用与前文一致的加权口径。

表 12 最终策略的五场客户端运行记录

运行序号	成功清除数	虚拟总时间 / s	平均时间 / (s/源)	程序时间 / s
1	16	2783.379572	173.961223	3.512500
2	10	2694.466566	269.446657	2.401488
3	15	3223.130037	214.875336	2.613757
4	10	2447.553722	244.755372	2.108452
5	12	2883.703512	240.308626	2.534841

五场记录的官方核验标志均为否，官方总源数、案例编码及导出日志字段为空。因此，表 12 确认的是客户端记录到的成功次数与时间，尚不能据此报告官方核验的清除比例，亦不能与不同场景的旧演练成绩计算配对改善。源数达到上界的一场在程序内按 16 源上界结束，其余场次按覆盖任务及轨迹清空结束；这些状态仍与独立官方结果核验属于不同证据。

题面要求的三次正式测试已于 2026 年 9 月 13 日全部完成，逐场案例编码、清除数、平均定位清除时间与程序运行时间列于表 13；原名导出的加密日志随支撑材料提交。正式测试界面不显示真值总数，表 13 只报告实际清除数与时间，不推算清除比例。

表 13 问题三正式测试结果

测试	案例编码	清除数	平均定位清除时间	程序运行时间
测试 1	BMW-GPHM-VG7 T-3CHM	13	234.42 s	2.749 s
测试 2	BA3P-CEGF-RT5R- CFB4	14	237.04 s	3.159 s
测试 3	D9M2-499N-PWQR- 5ZZJ	11	245.43 s	2.665 s

注：程序运行时间由成功/enter 与/exit 响应的 real_timestamp_ms 差值估计，时间戳分辨率 1 ms；为公开接口重建值，并非官方界面转录值。

6 问题四：混合干扰源的自动搜索、定位与清除

6.1 模型建立

问题四的目标区域中同时存在全向源与定向源。定向源的发射方向 δ 未知，其有效覆盖角度为 δ 两侧各 90°。设源位置为 g 、有效接收半径为 R ，则在测点 p 能够接收到该源信号的充要条件为： $\|p-g\| \leq R$ ，且 $(p-g) \cdot u(\delta) \geq 0$ ，其中 $u(\delta)$ 为发射方向的单位向量；全向源相当于第二个条件恒成立。

定向方向与接收半径均未知，因此 `no_signal` 存在两类歧义：测点可能位于发射扇区之外，也可能超出有效接收半径；此外频道无源或已被清除同样返回 `no_signal`。单一 `no_signal` 不能排除任何圆盘或半平面，这正是问题三第 5.2 节无信号排除规则在问题四失效的原因。问题四因此只利用正向观测：`direction` 给出以测点为中心、宽度 $2\alpha_c$ 的示向楔形，`near` 给出距离不超过 5 m 的直接清除机会；位置可行集 P_c 仍按楔形逐次求交更新，首次观测后用目标圆域、1500 m 接收圆与示向楔形构造先验外包络。

本策略不直接用负观测裁剪位置集，因此搜索完整性由扫描覆盖独立保证。最终采用 22 点环布局：原点、半径 998 m 内环 7 点和半径 1848 m 外环 14 点（图 18(b)）。对覆盖目标圆域的三角片 C，选取到其三个顶点均在接收半径内的扫描点集 N_C ；若 C 包含于 $\text{conv}(N_C)$ ，则片内任意源 g 均可被发现。证明如下：距离的凸性保证 g 到每个所选扫描点不超过顶点距离上界；若某发射方向 u 的正向闭半平面内没有所选点，则全部 $(p-g) \cdot u < 0$ ，但 g 为这些点的凸组合，故其加权和应为 0，产生矛盾。因此任意发射方向至少有一个点同时满足距离与可见性要求。

实现分为在线浮点检查与离线精确复核。在线程序以 999.99 m 筛选邻点，自适应二分外接 96 边形的三角片，并在启动时拒绝未通过的布局。离线程序对外域再增加 10^{-6} m 径向裕量，以有理数精确检验外域包含、三角片凸包含及距离平方 $\leq 1000^2 \text{ m}^2$ ；细分中点也用精确有理数表示，保证剖分无缝。两步均得到 2831 片通过、共 5566 次检查、最大深度 12。999.99 m 是浮点筛选裕量，最终精确证明使用题定下界 1000 m；两种验证的程序及结果均随支撑材料提供。

清除环节与问题三共用几何保证：包围圆半径 $r_c \leq 19.9 \text{ m}$ 时到圆心认证清除； $r_c \leq 60 \text{ m}$ 且尚未试清时，允许在圆心试清一次，仅 `success` 计入清除。若仍未清除，则可转入完整矩形回退：在定位多边形的直径坐标系中，以边长不超过 28 m 的矩形单元覆盖其外包矩形，在每个单元中心尝试。单元内任意源到中心至多 $\sqrt{(14^2+14^2)} = 19.80 \text{ m} < 20 \text{ m}$ ；所有中心按最近邻和 2-opt 顺序访问，直至清除成功。该有限回退不依赖再次收到射频信号，完整性结论仍以题定模型及充足运行预算为前提。

时间指标沿用式(25)、(26)：移动速度 5 m/s，每次检测 5 s，仅频道实际改变时另加 1 s，成功清除 5 s、失败 3 s。优化目标是在全部清除的前提下减少总时间。

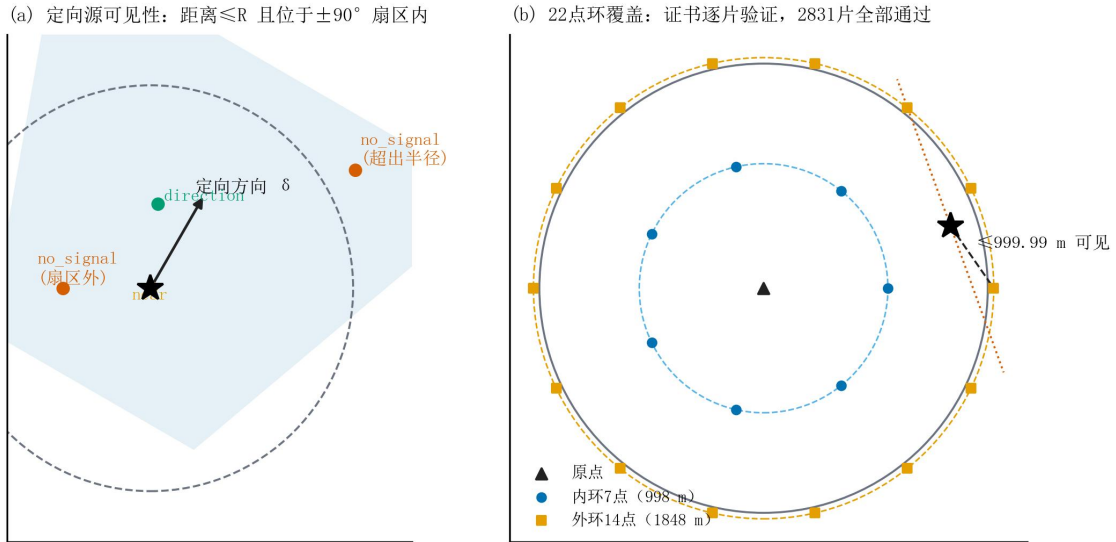


图 18 定向源可见性与 22 点环覆盖证书

注：no_signal 存在扇区外与超出半径两类歧义；右图圆点为扫描停点，虚线圆为内环（998 m）与外环（1848 m）。

6.2 策略与算法求解

问题四在问题三的在线状态机上作三处修改。第一，频道仍划分为未发现 U、已发现未清除 H 与已清除 E 三个集合，但 H 中频道只保存正向观测得到的可行多边形

P_c 与最近测点，no_signal 不再参与可行集更新，只跳过该频道。

第二，调度器采用延迟调度：对仅获得一次示向、包围圆半径大于 60 m 且仍有未访问扫描点的目标，暂不纳入定位路线，先把机器狗派往覆盖扫描或其它已成熟目标；在这些停点执行共享交叉测向——当已知目标的包围圆半径大于 19.9 m、与当前停点距离小于 1200 m 且相对其最近测点转过足够角度时，顺带复测该频道，用新的楔形与 P_c 求交。目标一旦获得两次以上示向或包围半径不超过 60 m，立即并入联合路线。该机制把定向源的信息获取安排在顺路的扫描停点，减少专程折返；由于只使用正向观测，它不依赖问题三中基于负观测的排除规则。

第三，扫描节点与待处理目标中心共同参与开放路径规划，以多起点最近邻和 2-opt 取得访问次序，每次仅执行首站。定位先检查 19.9 m 认证清除及 60 m 以内的一次试清；若仍未成功，最终 optical 模式在已有至少两次示向、或已执行两轮补测时，直接转入矩形光学回退。补测使用最近示向的横向偏移或首次示向的插值点；通用基类 8 轮上限并非最终版本通常采用的回退阈值。每个扫描点检测所有未发现频道，并共享测向。清除 16 个不同频道可按总数上界提前结束；否则必须完成全部扫描且已知目标清空。参数见表 14。

表 14 问题四最终策略的主要参数

参数	采用值	作用
扫描布局	22 点环（原点+内环 7 点 998 m +外环 14 点 1848 m）	覆盖证书的扫描点集
覆盖证书	浮点检查+离线有理数复核	2831 片通过；精确距离 ≤ 1000 m
证书距离上限	浮点筛选 999.99 m	精确复核 1000 m 题定下界
延迟调度阈值	单次示向且包围半径 >60 m	推迟到共享观测后再定位
共享测向条件	半径 >19.9 m、停点距中心 <1200 m	顺路复用观测
认证清除半径	19.9 m	在最小包围圆圆心处清除
光学尝试半径	60 m，每频道至多一次	仅 success 计清除
矩形回退网格	≤ 28 m	最坏距离 19.80 m <20 m
最终回退触发	已有 ≥ 2 次示向或已补测 2 轮	先认证/一次试清，再转矩形回退
建模半界 α_c	1.005°	覆盖题定误差与舍入裕量

6.3 演练测试及结果分析

问题四分别以本地合成场景和官方模拟器检验。同场景配对比较 25 点环基线与 22 点环延迟调度策略，结果见表 15。开发种子 1—40 用于选择参数；新留出种子 40001—40100 及八类压力种子 201—210 用于后续评价。两策略在 220 个场景、共 440 次运行中均全部清除，时间差异不由漏清产生。新留出 100 场中 77 场更快，平均降低 3.97%，配对重抽样 95%区间为[2.72%,5.14%]；失败试清从 2456 次减至 504 次。压力集平均退化 2.36%，其中边缘与相切组分别退化 12.33%、13.17%，因此平均收益不能外推到每类布局。

表 15 问题四同场景配对验证（两策略均全部清除）

集合	场景/源数	25 点基线 s/源	22 点策略 s/源	时间降低
开发集	40/526	468.508	456.263	2.61%
新留出	100/1321	468.525	449.902	3.97%
压力集	80/1008	531.926	544.504	-2.36%

官方模拟器另完成 5 场演练，逐场清除比例及时间见表 16。案例构成与本地合成场景不同，两类均值仅分别报告。

表 16 问题四官方模拟器演练结果（5 场，非正式测试）

案例编码	源数（全向/ 定向）	清除数	清除比例	平均时间 / (s/源)	客户端墙钟 /s	条带回退
TNU2-3QVU -7TQ5-EYY Y	15（12/3）	15	15/15	353.39	11.16	1
CTUE-84C5- V4P2-T9MG	10（0/10）	10	10/10	738.10	7.57	1
RBE4-DUCV -QFFJ-YSGR	14（3/11）	14	14/14	435.88	6.43	1
VMQJ-EFAQ -CBQS-R4U 7	16（13/3）	16	16/16	315.50	5.94	1
QTKQ-7845- FHE2-8BFJ	14（1/13）	14	14/14	460.70	7.48	3

5 场演练共清除 69/69 个源，清除比例 100%；加权平均 438.87 s/源，逐场范围 315.50—738.10 s/源，客户端墙钟 5.94—11.16 s。纯定向案例 CTUE 耗时最高，但五场数据不足以建立定向比例与耗时的函数关系。图 19 展示初版和最终版两批演练：前者 5 场 67 个源、均值 1250.14 s/源，后者 5 场 69 个源、均值 438.87 s/源。两批随机案例不同，差异同时包含算法与场景因素，只作描述对照。

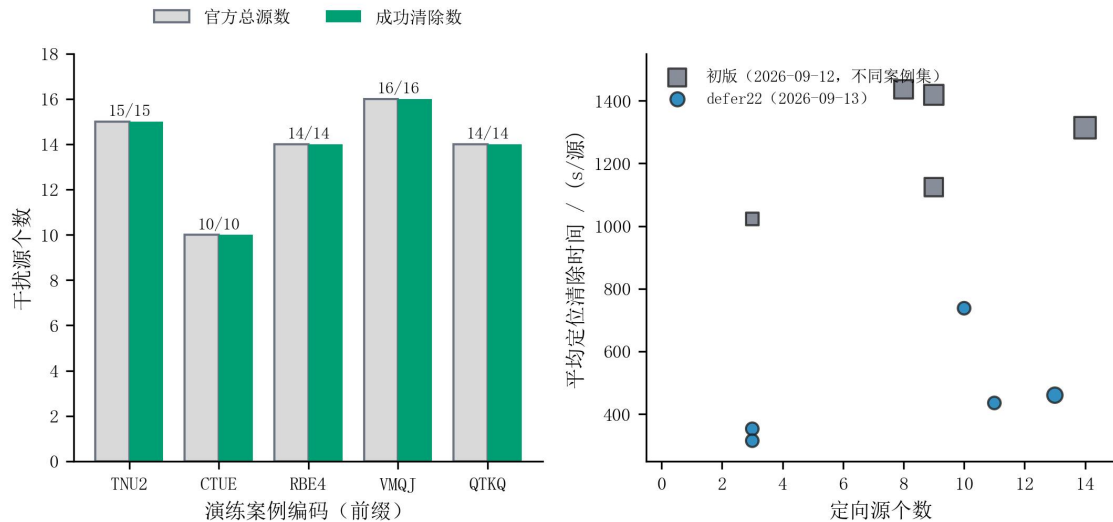


图 19 问题四两批官方演练的清除结果与定向源—耗时关系

注：均为演练结果，非正式成绩；不同批次为不同随机案例，仅作参考对照；点大小为条带回退次数。

如图 20 展示纯定向案例 CTUE 的实际动作轨迹及逐场动作计时。该案例 10 个源全部清除，16 次清除尝试中 6 次失败，矩形回退 1 次。五场中移动占 72.5%—79.7%，检测占 15.7%—22.0%，切换占 2.9%—4.2%，清除占 0.9%—2.1%，分解残差为 0 s。移动仍是主要耗时来源；算法平均改进的比较依据是表 15 的同场景留出试验，不能由两批不同官方演练均值直接归因。

以上均为演练结果。演练数据说明策略在混合源场景中能够全部清除并在 20 min 程序预算内完成；正式成绩见表 17。

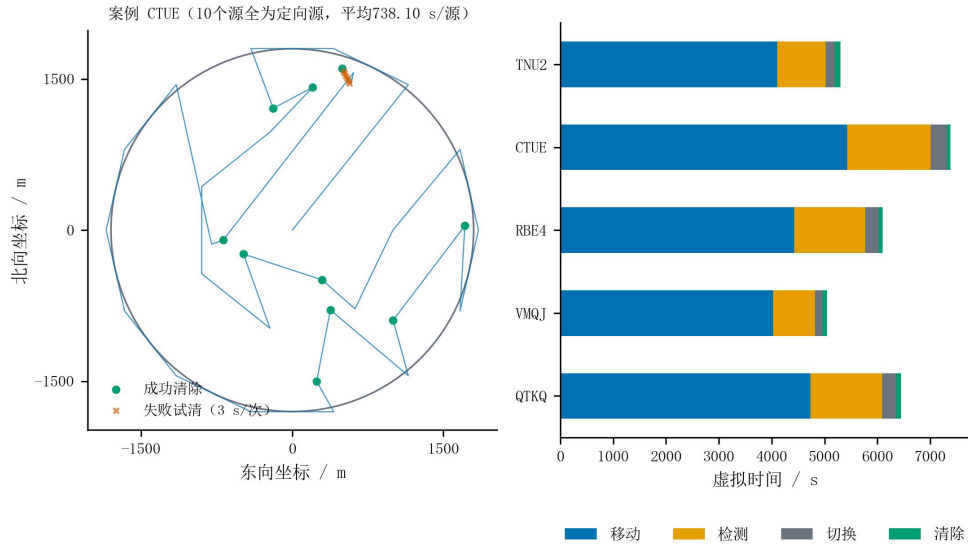


图 20 问题四演练案例 CTUE 的动作轨迹与逐场时间分解
注：源自官方演练日志；时间分解残差为 0 s。

6.4 正式测试结果

题面要求问题四进行三次正式测试，并在论文中按题面表 1 的格式报告案例编码、清除源个数、平均定位清除时间与程序运行时间，同时导出加密日志放入支撑材料。三次正式测试已于 2026 年 9 月 13 日完成，成绩列于表 17，原名加密日志随支撑材料提交。其中两场清除 16 个、达到题面上限，另一场按覆盖完整性状态完整结束。

正式测试界面不显示案例真值总数，表 17 仅报告实际清除数与时间，不推算未知总数下的清除比例。按表中数值和清除数加权，三场描述均值约 407.20 s/源；该均值只概括本组三场，不能与问题三不同场景的均值作算法优劣推断。原名加密日志与公开记录见支撑材料。

表 17 问题四正式测试结果

测试	案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
测试 1	ZN5C-TH9Z-TDKY-DD3R	16	336.07 s	5.831 s
测试 2	UVDP-3HKV-TEZF-RCBD	10	592.76 s	6.127 s
测试 3	94M3-RY77-35PX-MAPV	16	362.36 s	6.085 s

注：程序运行时间由成功/enter 与/exit 响应的 real_timestamp_ms 差值估计，时间戳分辨率 1 ms；为公开接口重建值，并非官方界面转录值。

7 模型评价与适用边界

7.1 问题一、二模型的优点

首先，误差处理与题面一致。观测始终以有界角误差集合进入模型，避免在缺少概率依据时人为引入噪声分布，并能直接检查真实位置是否被保留。其次，区域状态、直径与包围圆具有清晰的几何含义：空集暴露观测不一致，无界表示信息不足，直径衡量两点差异，包围圆支持实际清除点的安全判断。

再次，第二点选择利用“首次已接收到信号”这一条件信息，通过径向端点与角向极值把连续接收约束化为有限判据。保证接收与数值优选分层，能够准确区分证明成立的性质与实验支持的表现；典型点对照也解释了沿示向前进和无保证横移各自的问题。

7.2 问题一、二模型的局限

顶点枚举和包围圆枚举以透明性为主要考虑，检测数增大时运算量会上升；近平行的观测边界还可能放大浮点误差。固定地点误差使同点重复检测无益，模型也不能通过增加同点次数人为缩小置信范围。若真实测角误差超过设定界，包含保证将失效，需要重新检查输入与误差模型。

第二点的保证区域忽略了目标圆域对首次扇区的部分截断，因此可能排除某些实际可安全使用的位置。有限场景评分、50 m 候选网格和局部 25 m 加密均不能替代连续全局最优证明。外切多边形虽然保持位置包含，但引入了保守性；进一步提高边数应与运算代价和实际设备精度共同权衡。

7.3 问题三策略的优点与局限

第三题把扫描与定位共享的停点信息纳入同一任务路线，减少逐源独立处理造成的重复移动。可见性约束利用固定接收半径产生的信息；圆周覆盖与径向证明使扫描点具有位置调整空间，同时保留连续搜索范围。清除成功、普通试探失败与认证异常分开处理，能避免把失败动作计入清除数。

该策略仍以名义圆心和局部路径长度评价任务，不能保证对每场源布局都更快；边缘压力组及 16 个留出退化场景说明了这一限制。100 m 先导、共享阈值和 50 m 试清参数来自开发集，可能随目标分布变化而改变收益。全部清除的几何论证依赖题定全向机制、观测界和足够运行预算，有限合成验证与客户端记录均不能替代正式测试。

7.4 问题四模型评价

问题四沿用集合定位与认证清除内核：22 点环布局的覆盖证书把发现完整性化为可验证的凸包含关系，矩形回退把清除完整性化为不依赖后续信号的有限扫描（最坏距离 $19.80\text{ m} < 20\text{ m}$ ）；延迟调度与共享交叉测向在不使用负观测的前提下减少重复往返。正负观测的严格区分避免把问题三的 `no_signal` 排除规则错误移植到定向源上。

本策略未将负观测转化为位置与方向的联合约束，因而放弃了可能有用的析取信息；可行集主要靠正向观测收缩，定向源定位耗时较大。边缘与相切压力组相对 25 点基线出现退化，说明环覆盖加延迟调度并非逐类占优。问题三的 100 m 先导及全向可见性约束未直接移植到问题四，两问的性能仍受不同观测机制制约。此外正式测试仅 3 场且真值不可见，不能据此建立定向比例与时间的关系。

后续改进可将定向方向与位置联合估计纳入可行集，例如用同一频道多次 `direction` 的扇区一致性反推方向区间；或利用 `no_signal` 的扇区外与超距两类事件构造析取约束，以减少矩形回退；也可把延迟调度扩展为基于信息收益的最优停点选择。

8 结论

8.1 交会定位与检测点选择结论

问题一可通过半平面交点枚举和衰退方向判别恢复区域状态，在非空有界时取最大顶点对距离得到定位直径。实际 2° 楔形反例的直径为 28.96681 m ，直径圆仍漏覆盖 0.26549 m ，故题设圆不能保证覆盖定位区域；清除保证应以最小包围圆半径为依据。

问题二宜先构造连续保证接收区，再在其中比较有限场景下的定位直径。标准算例在 50 m 网格上选择 $(800, -600)\text{ m}$ ，有限场景最坏直径 112.14914 m ；10% 容忍度得到 24 个优选点，局部加密得到 $(850, -525)\text{ m}$ 及 111.89311 m 。上述数值回答了给定算例的选点与候选区域问题，其适用范围限定于所计算的网格和场景。

8.2 全向源自动搜索与清除结论

问题三采用原点及外圈连续覆盖、持久化集合定位、100 m 共享先导和覆盖约束下的弹性路径规划，结合认证清除与有限回退形成闭环策略。开发、压力与新留出三集合共 190 场、2431 个源均全部清除；最具独立评价意义的新留出集为 100 场、1265 个源，平均 233.723 s/源 ，比相同场景的上一版降低 5.77%。

改进主要来自移动距离下降，而非减少必须清除的目标。边缘压力组略有退化，整体尚未达到 200 s/源，也未证明全局时间最优。五场客户端记录合计 63 次成功清除、平均 222.734 s/成功清除源；因官方总源数与正式案例日志尚未核验，本文不将其表述为官方清除率或正式测试成绩。问题三三次正式测试已于 2026 年 9 月 13 日完成，实际清除 13、14、11 个，平均 234.42—245.43 s/源（表 13），原名加密日志随支撑材料提交。

8.3 混合源搜索与清除结论

针对混合干扰源，本文建立了定向可见性模型，采用 22 点环覆盖证书（2831 个三角片全部通过）保证发现完整性，以延迟调度与共享交叉测向压缩定位时间，并用最小包围圆认证清除与矩形回退保证清除完整性。

本地合成三集共 220 场评估全部清除；最具独立评价意义的新留出集为 100 场 1321 个源，平均 449.902 s/源，较第一轮优化版降低 3.97%（配对 bootstrap 95% 区间 [2.72%, 5.14%]）。官方演练 5 场 69 个源全部清除，平均 438.87 s/源；初版演练 5 场平均 1250.14 s/源，作为不同案例集的描述对照。

问题四三次正式测试已于 2026 年 9 月 13 日完成：清除 10—16 个，平均 336.07—592.76 s/源，程序运行 5.831—6.127 s（响应端点估计）（表 17）；其中两场达到 16 个的题面上限。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模表达整理、代码调试与结果核对、图表整理、论文草稿撰写和提交材料审查，详细使用情况见支撑材料。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题：无线电干扰源的快速自动定位与清除[Z]. 2026.
- [2] OSHMAN Y, DAVIDSON P. Optimization of observer trajectories for bearings-only target localization[J]. IEEE Transactions on Aerospace and Electronic Systems, 1999, 35(3): 892–902. DOI: 10.1109/7.784059.

附录

附录 A 支撑材料目录与复现入口

支撑材料按问题分级组织。问题一、二：问题一与问题二/code 及 results/q1_*, q2_*；问题三最终入口：问题三/code/run_robot.py；问题四最终入口：问题四/code/run_q4.py。下列清单与压缩包逐项一致；核心源程序的注释选读见附录 B，全部可运行源程序保存在压缩包内。

在解压根目录执行 `python verify_submission.py` 可核验包内 SHA-256，并隔离重放问题一楔形反例与最小包围圆、问题二两种网格最优值、问题三策略回归测试、问题四 22 点布局离线有理数复核（2831 片通过）与私有模拟 HTTP 协议自测，全部通过时末行输出 PASS。

正式测试六场逐场客户端记录位于正式测试/客户端记录/，六份原名加密日志位于正式测试/加密日志/，成绩与计时证据见同目录的 `official_results_to_fill.csv` 与 `formal_timing_evidence.json`；论文表 13 与表 17 的程序运行时间由成功/enter 与/exit 响应服务器时间戳差估计，为公开接口重建值。

AI 工具使用详情.pdf

README_复现说明.md

requirements.txt

submission_manifest.json

verify_submission.py

正式测试/formal_timing_evidence.json

正式测试/official_results_to_fill.csv

正式测试/加密日志/formal-p3-1-BMWA-GPHM-VG7T-3CHM.jlog

正式测试/加密日志/formal-p3-2-BA3P-CEGF-RT5R-CFB4.jlog

正式测试/加密日志/formal-p3-3-D9M2-499N-PWQR-5ZZJ.jlog

正式测试/加密日志/formal-p4-1-ZN5C-TH9Z-TDKY-DD3R.jlog

正式测试/加密日志/formal-p4-2-UVDP-3HKV-TEZF-RCBD.jlog

正式测试/加密日志/formal-p4-3-94M3-RY77-35PX-MAPV.jlog

正式测试/客户端记录/q3_1_BMWA-GPHM-VG7T-3CHM/client.jsonl

正式测试/客户端记录/q3_1_BMWA-GPHM-VG7T-3CHM/summary.json

正式测试/客户端记录/q3_2_BA3P-CEGF-RT5R-CFB4/client.jsonl

正式测试/客户端记录/q3_2_BA3P-CEGF-RT5R-CFB4/summary.json

正式测试/客户端记录/q3_3_D9M2-499N-PWQR-5ZZJ/client.jsonl

正式测试/客户端记录/q3_3_D9M2-499N-PWQR-5ZZJ/summary.json

正式测试/客户端记录/q4_1_ZN5C-TH9Z-TDKY-DD3R/client.jsonl

正式测试/客户端记录/q4_1_ZN5C-TH9Z-TDKY-DD3R/summary.json

正式测试/客户端记录/q4_2_UVDP-3HKV-TEZF-RCBD/client.jsonl

正式测试/客户端记录/q4_2_UVDP-3HKV-TEZF-RCBD/summary.json

正式测试/客户端记录/q4_3_94M3-RY77-35PX-MAPV/client.jsonl

正式测试/客户端记录/q4_3_94M3-RY77-35PX-MAPV/summary.json

问题一与问题二/code/config.json

问题一与问题二/code/geometry.py

问题一与问题二/code/offline_sim.py

问题一与问题二/code/q1_example.json

问题一与问题二/code/robot_client.py

问题一与问题二/code/solve_q1.py

问题一与问题二/code/solve_q2.py

问题一与问题二/code/strategy.py

问题一与问题二/code/study.py

问题一与问题二/code/test_core.py

问题一与问题二/results/q1_constraint_sequence.csv

问题一与问题二/results/q1_observations.csv

问题一与问题二/results/q1_vertices.csv

问题一与问题二/results/q2_candidates.csv

问题一与问题二/results/q2_comparison.csv

问题一与问题二/results/q2_eta_sensitivity.csv

问题一与问题二/results/q2_local_refinement.csv

问题一与问题二/results/synthetic_runs.csv

问题三/code/baseline_breakthrough.py

问题三/code/baseline_gemini.py

问题三/code/baseline_original.py

问题三/code/benchmark.py

问题三/code/config.json

问题三/code/coverage_exact.py

问题三/code/frozen_config.json

问题三/code/geometry.py
 问题三/code/offline_sim.py
 问题三/code/prototype_v1.py
 问题三/code/prototype_v10.py
 问题三/code/prototype_v2.py
 问题三/code/prototype_v3.py
 问题三/code/prototype_v4.py
 问题三/code/prototype_v5.py
 问题三/code/prototype_v6.py
 问题三/code/prototype_v7.py
 问题三/code/prototype_v8.py
 问题三/code/prototype_v9.py
 问题三/code/robot_client.py
 问题三/code/robot_core.py
 问题三/code/run_robot.py
 问题三/code/strategy.py
 问题三/code/test_strategy.py
 问题四/code/benchmark.py
 问题四/code/coverage.py
 问题四/code/coverage_certificate.py
 问题四/code/frozen_config.json
 问题四/code/geometry.py
 问题四/code/http_selftest.py
 问题四/code/mock_simulator.py
 问题四/code/offline_sim.py
 问题四/code/results/pi_exact_coverage_audit.py
 问题四/code/ring_certificate.py
 问题四/code/robot_client.py
 问题四/code/robot_core.py
 问题四/code/routing.py
 问题四/code/run_q4.py
 问题四/code/selected_new.json
 问题四/code/strategy.py
 问题四/code/strategy_active.py
 问题四/code/strategy_explore.py
 问题四/code/strategy_final.py
 问题四/code/strategy_particle.py
 问题四/code/strategy_ring21.py
 问题四/code/strategy_selected.py

附录 B 核心源程序选读（附注释）

以下按“问题一/二→问题三→问题四→公共接口”的顺序选读承载模型与证明的代码：可行域构造、覆盖与清除证书、路径规划与主循环；实验脚手架、绘图与审计脚本不在选读之列。以“# 注：”开头的注释行为阅读辅助插入，代码本体与支撑材料中的原文件逐字一致。全部 51 份可运行源程序及运行配置见支撑材料压缩包（清单与 SHA-256 校验见附录 A 与 submission_manifest.json）。

B.1 问题一与问题二：可行域、直径与最小包围圆

问题一、二共享同一几何内核：把有界测角误差下的可能位置写成半平面交集，在此之上定义定位直径与最小包围圆，并把“保证接收”化为连续判据。

文件：问题一与问题二/code/geometry.py

选读说明：全部定位与覆盖计算的公共内核（全文）。

```

"""Bounded-bearing geometry. Coordinates m; public angles degrees. Standard library only."""
import itertools
import math

TOL = 1e-7

def add(a,b): return (a[0]+b[0],a[1]+b[1])
def sub(a,b): return (a[0]-b[0],a[1]-b[1])
def mul(a,t): return (a[0]*t,a[1]*t)
def dot(a,b): return a[0]*b[0]+a[1]*b[1]
def cross(a,b): return a[0]*b[1]-a[1]*b[0]
def dist(a,b): return math.hypot(a[0]-b[0],a[1]-b[1])
  
```

```

def unit(deg):
    t=math.radians(deg); return (math.cos(t),math.sin(t))
def bearing(a,b): return math.degrees(math.atan2(b[1]-a[1],b[0]-a[0]))%360

# 注: 把示向度  $\theta \pm \alpha$  写成两个闭半平面的交, 可行域即若干半平面的交集。
def wedge(s,theta,alpha=1.0):
    if not 0<alpha<90: raise ValueError('Require 0 < alpha < 90 degrees')
    lo,hi=unit(theta-alpha),unit(theta+alpha)
    n1,n2=(lo[1],-lo[0]),(-hi[1],hi[0])
    return [(n1[0],n1[1],dot(n1,s)),(n2[0],n2[1],dot(n2,s))]

def inside(p,hps,tol=TOL): return all(a*p[0]+b*p[1]<=c+tol for a,b,c in hps)

# 注: 单调链凸包; 剔除重复与共线点后输出逆时针顶点序列。
def hull(points):
    pts=sorted(points); unique=[]
    for p in pts:
        if not unique or dist(p,unique[-1])>TOL: unique.append(p)
    if len(unique)<3: return unique
    def half(seq):
        h=[]
        for p in seq:
            while len(h)>1 and cross(sub(h[-1],h[-2]),sub(p,h[-1]))<=1e-10: h.pop()
            h.append(p)
        return h
    return half(unique)[-1]+half(reversed(unique))[:-1]

# 注: 两两半平面求交得到候选顶点, 保留满足全部约束者再做凸包; 若存在满足全部半平面的方向向量则判为无界。
def intersection(observations,alpha=1.0):
    """Pure wedges, no artificial clipping box; statuses empty/unbounded/bounded."""
    hs=[h for x,y,t in observations for h in wedge((x,y),t,alpha)]
    if not hs: return {'status':'unbounded','vertices':[],'diameter':None}
    pts=[]
# 注: 枚举全部半平面对的点; 本题约束至多 32 条,  $O(n^2)$  已足够。
    for (a,b,c),(d,e,f) in itertools.combinations(hs,2):
        det=a*e-b*d
        if abs(det)<1e-12: continue
        p=((c*e-b*f)/det,(a*f-c*d)/det)
        if inside(p,hs): pts.append(p)
    if not pts: return {'status':'empty','vertices':[],'diameter':None}
    vs=hull(pts)
    for a,b,_ in hs:
        for v in [(b,-a),(-b,a)]:
            if all(d*v[0]+e*v[1]<=1e-12 for d,e,_ in hs):
                return {'status':'unbounded','vertices':vs,'diameter':None}
    return dict(status='bounded',vertices=vs,**diameter_info(vs))

# 注: 凸多边形直径由某个顶点对达到; 同时给出“以直径中点为圆心”的覆盖缺陷 excess。
def diameter_info(vs):
    if not vs: raise ValueError('Empty polygon')
    a,b=max(itertools.product(vs,repeat=2),key=lambda pair:dist(*pair))
    d=dist(a,b); c=mul(add(a,b),0.5)
    excess=max(dist(v,c)-d/2 for v in vs)
    return {'diameter':d,'diameter_pair':[a,b],'diameter_center':c,
            'diameter_circle_covers':excess<=TOL,'circle_excess':excess}

# 注: 最小包围圆由顶点集的 1 点/2 点/3 点子集决定: 枚举顶点、两点中点与三点外心即可。
def enclosing_circle(vs):
    if not vs: raise ValueError('Empty polygon')
    best=(None,math.inf)
    def consider(c,r):
        nonlocal best
        if r<best[1] and all(dist(v,c)<=r+TOL for v in vs): best=(c,r)

```

```

for v in vs: consider(v,0)
for a,b in itertools.combinations(vs,2):
    c=mul(add(a,b),0.5); consider(c,dist(a,b)/2)
for a,b,c in itertools.combinations(vs,3):
    ab,ac=sub(b,a),sub(c,a); den=2*cross(ab,ac)
    if abs(den)<1e-12: continue
    bb,cc=dot(ab,ab),dot(ac,ac)
    o=add(a,((bb*ac[1]-cc*ab[1])/den,(ab[0]*cc-ac[0]*bb)/den))
    consider(o,dist(o,a))
if best[0] is None: raise ArithmeticError('No enclosing circle')
return best

# 注: Sutherland-Hodgman 半平面裁剪, 每次裁剪后重算凸包消除退化点。
def clip(poly, hp):
    if not poly: return []
    a,b,c=hp; out=[]
    for p,q in zip(poly, poly[1:]+poly[0]):
        fp=a*p[0]+b*p[1]-c; fq=a*q[0]+b*q[1]-c
        ip,iq=fp<=TOL, fq<=TOL
        if ip: out.append(p)
        if ip!=iq:
            t=fp/(fp-fq); out.append(add(p,mul(sub(q,p),t)))
    return hull(out)

def clip_wedge(poly, s, theta, alpha=1.0):
    for h in wedge(s, theta, alpha): poly=clip(poly, h)
    return poly

def circle_hps(c, r, n=128):
    return [(u[0], u[1], dot(u, c)+r) for u in (unit(360*j/n) for j in range(n))]

# 注: 首次观测的可行域=全平面n1800 m圆盘(128边外切)n接收半径≤1500 mn误差楔形; 外切多边形只放大可行域, 保证不误删真解。
def prior(s, theta, alpha=1.0, n=128):
    # Outer tangent polygon: every real point in both circles remains feasible.
    poly=[(-1800, -1800), (1800, -1800), (1800, 1800), (-1800, 1800)]
    for h in circle_hps((0,0), 1800, n)+circle_hps(s, 1500, n)+wedge(s, theta, alpha):
        poly=clip(poly, h)
    return poly

def local_to_global(s, theta, p):
    u=unit(theta); v=(-u[1], u[0]); return add(s, add(mul(u, p[0]), mul(v, p[1])))

# 注: 连续保证接收判据: 对  $r \in [5, 1500]$  与角度  $\in [-\alpha, \alpha]$  的连续最不利情形取投影极小值; 极值只可能在角度端点或与角平分线反向处。
def guaranteed(b, alpha=1.0):
    """Continuous guarantee over  $r$  in  $[5, 1500]$ , angle in  $[-\alpha, \alpha]$ ."""
    ang=math.degrees(math.atan2(b[1], b[0]))
    opposite=(ang+360)%360-180
    ts=[-alpha, alpha]
    if -alpha<=opposite<=alpha: ts.append(opposite)
    minimum=min(dot(b, unit(t)) for t in ts)
    return all(dot(b, b)+r*r-2*r*minimum<=max(1000, r)**2+TOL for r in (5, 1000, 1500))

# 注: 有限场景最坏定位直径: 对每个采样源位置, 以“真方位+误差角”的楔形再裁剪可行域, 取裁剪后直径的最大值作为评分。
def sampled_score(s, theta, b, sources, errors, alpha=1.0, poly=None):
    if poly is None: poly=prior(s, theta, alpha)
    worst=0.0
    for g in sources:
        if dist(b, g)<=5: continue
        t=bearing(b, g)
        for e in errors:
            p=clip_wedge(poly, b, t+e, alpha)

```

```

        if not p: raise ArithmeticError('Sample excluded by its own wedge')
        worst=max(worst,diameter_info(p)['diameter'])
    return worst

# 注：有限源场景构造：按接收半径与角度离散（问题二评分与灵敏度分析使用）。
def source_samples(s,theta,alpha=1.0,dense=False):
    rs=[6]+list(range(50 if dense else 100,1501,50 if dense else 100))
    ts=[-alpha,-alpha/2,0,alpha/2,alpha] if dense else [-alpha,0,alpha]
    return [g for r in rs for t in ts if dist((0,0),g:=local_to_global(s,theta,mul(unit(t),r)))<=1800+TOL]
文件：问题一与问题二/code/solve_q1.py
选读说明：问题一命令入口（全文）：输入观测文件，输出可行域状态、顶点与最小包围圆。
"""Generic Q1 command: python B 题/code/solve_q1.py input.json --output answer.json"""
import argparse,json,math
from pathlib import Path
from geometry import intersection,enclosing_circle

def main():

p=argparse.ArgumentParser();p.add_argument('input');p.add_argument('--output',default='q1_answer.json');a=p.parse_args()

data=json.loads(Path(a.input).read_text(encoding='utf-8-sig'));obs=data['observations'];alpha=float(data.get('alpha_deg',1))
    if any(len(row)!=3 or any(isinstance(x,bool) or not isinstance(x,(int,float)) or not math.isfinite(x) for x in row) for row in obs):raise ValueError('Each observation must be finite [x,y,bearing_deg]')
# 注：按题意区分有界/无界/空集三种状态；只有有界时才继续计算直径与覆盖。
    result=intersection(obs,alpha)
# 注：最小包围圆给出“任意可能源位置与清除点距离上界”的认证，是问题一能否直接清除的判据。
    if
result['status']=='bounded':result['mec_center'],result['mec_radius']=enclosing_circle(result['vertices'])

Path(a.output).write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8');print(json.dumps(result,ensure_ascii=False))
if __name__=='__main__':main()
文件：问题一与问题二/code/study.py
选读说明：问题二求解与结果表生成（选摘网格枚举与评分的核心过程）。
def csv_write(path,rows):
    path.parent.mkdir(parents=True,exist_ok=True)
    with path.open('w',encoding='utf-8-sig',newline='') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)

# 注：问题二主过程：在同一网格上枚举第二检测点，保留连续保证接收者。
def solve_q2(s=(0,0),theta=0,step=50):
    sources=source_samples(s,theta);poly=prior(s,theta);rows=[]
    if not sources:raise ValueError('First observation has no sampled source within arena')
# 注：网格范围  $0\leq x\leq 1000$ 、 $-800\leq y\leq 800$ ，步长可调（实验用 50 m，局部加密 25 m）。
    for x in range(0,1001,step):
        for y in range(-800,801,step):
# 注：先做连续保证接收过滤，不满足者不参与评分。
            if not guaranteed((x,y)):continue
            b=local_to_global(s,theta,(x,y))
# 注：对每个候选点用有限源场景与误差角（ $-1^\circ, 0^\circ, +1^\circ$ ）计算最坏定位直径。
            score=sampled_score(s,theta,b,sources,[-1,0,1],poly=poly)

rows.append(dict(local_x=x,local_y=y,x=b[0],y=b[1],worst_diameter_m=score,move_m=math.hypot(x,y)))
    # 1e-8 m tie resolution avoids floating-point asymmetry of reflected cases.
# 注：排序键：最坏直径优先、移动距离其次；1e-8 舍入避免镜像情形的浮点不对称。
    best=min(rows,key=lambda r:(round(r['worst_diameter_m'],8),r['move_m'],r['local_x'],r['local_y']))
# 注：10% 优选格点：评分不超过最优值 110% 的候选集合。
    for r in rows:r['candidate_10pct']=r['worst_diameter_m']<=1.1*best['worst_diameter_m']
    return rows,best

```

B.2 问题三：集合定位与覆盖约束路径规划

问题三按“先保证发现、再保证清除、最后优化路径”组织：100 m 单元覆盖账本与圆周覆盖证书保证发现；19.9 m 最小包围圆认证与有限条带回退保证清除；目标与扫描点统一进入开放路径规划。

文件：问题三/code/robot_core.py

选读说明：问题三/四共用的底层动作与认证清除内核（选摘回退、局部化与动作包装）。

■ 有限条带回退（保证“模型—接口一致”前提下的完备性）

```
def fallback_points(s, theta):
```

注：w=1500·sin(α)：以首观测点为顶点、张角 2α 的可见扇区横向半宽。

```
    w = 1500 * math.sin(math.radians(ALPHA))
```

注：沿 1500 m 纵深每 28 m 布一点（双向蛇形），域内任意点到某点距离不超过 20 m。

```
    xs = list(range(0, 1500, 28)) + [1500]
```

```
    return [local_to_global(s, theta, (x, y))
```

```
            for y, xx in [(-w / 2, xs), (w / 2, list(reversed(xs)))] for x in xx]
```

■ 认证清除与局部定位主循环

注：核心循环：观测—裁剪可行域—认证；半径≤19.9 m 时直接清除。

```
def localize_poly(strategy, ch, poly, s0, theta0, last, last_theta, viewpoint=None):
```

```
    """Iteratively shrink `poly` with further bearings, then certified-clear.
```

```
    viewpoint(poly, s0, theta0, last, last_theta) -> next measurement point.
```

```
    """
```

```
    for _ in range(4):
```

```
        if not poly:
```

```
            raise RobotError('Inconsistent bearing polygon')
```

注：最小包围圆半径≤19.9 m 即满足题定“20 m 内清除成功”的几何认证。

```
        c, r = enclosing_circle(poly)
```

```
        if r <= CLEAR_SAFE:
```

```
            out = strategy.action('clear', c, ch)
```

```
            if out['clear_result'] != 'success':
```

```
                raise RobotError('Certified clear failed')
```

```
            return
```

```
        if viewpoint is None:
```

```
            p = default_viewpoint(poly, s0, theta0, last, last_theta)
```

```
        else:
```

```
            p = viewpoint(poly, s0, theta0, last, last_theta)
```

```
            res = strategy.action('measure', p, ch)
```

注：near：目标在 5 m 内直接清除，无需进一步定位。

```
            if res['measure_result'] == 'near':
```

```
                hit = strategy.action('clear', p, ch)
```

```
                if hit['clear_result'] != 'success':
```

```
                    raise RobotError('near clear failed')
```

```
                return
```

```
            if res['measure_result'] == 'no_signal':
```

```
                break
```

注：追加示向后用新楔形继续裁剪可行域。

```
            poly = clip_wedge(poly, p, res['svd_deg'], ALPHA)
```

```
            last, last_theta = p, res['svd_deg']
```

```
            strategy.fallback_count += 1
```

注：迭代上限内未认证则转条带回退（有限步保证清除）。

```
            for p in fallback_points(s0, theta0):
```

```
                if strategy.action('clear', p, ch)['clear_result'] == 'success':
```

```
                    return
```

```
            raise RobotError('Complete strip sweep failed: model/interface inconsistent')
```

```
def localize(strategy, ch, s, res, viewpoint=None):
```

```
    """Entry point used by a single fresh observation."""
```

```
    if res['measure_result'] == 'near':
```

注：near 直接清除；否则以首个楔形初始化可行域进入局部化。

```
        out = strategy.action('clear', s, ch)
```

```
        if out['clear_result'] != 'success':
```

```
            raise RobotError('near then clear failed')
```

```

        return
    theta = res['svd_deg']
    poly = prior(s, theta, ALPHA)
    localize_poly(strategy, ch, poly, s, theta, s, theta, viewpoint=viewpoint)
■ 统一动作包装与预算检查
    def action(self, kind, p, ch):
# 注：每个动作先做现实时间软截止与虚拟时间预算检查。
        left = self.client.time_left()
        if left is not None and left < REAL_MIN:
            raise BudgetStop('real_time_budget')
# 注：按“当前虚拟时间+移动耗时+6 s”预判预算，超限即抛出 BudgetStop 优雅终止。
        if self.client.virtual_time + dist(self.client.position, p) / 5 + 6 > VIRTUAL_BUDGET:
            raise BudgetStop('virtual_time_budget')
        status, res = getattr(self.client, kind)(p[0], p[1], ch)
        if status != 200 or not isinstance(res, dict) or res.get('accepted') is not True:
            raise RobotError('Rejected action: %s %s' % (status, res))
        key = 'measure_result' if kind == 'measure' else 'clear_result'
        allowed = ('direction', 'near', 'no_signal') if kind == 'measure' else ('success',
'no_target_in_range')
        if res.get(key) not in allowed:
            raise RobotError('Invalid result code: ' + repr(res))
        if kind == 'measure' and res[key] == 'direction':
            angle = res.get('svd_deg')
            if isinstance(angle, bool) or not isinstance(angle, (int, float)) \
                or not math.isfinite(angle) or not 0 <= angle < 360:
                raise RobotError('Invalid bearing: ' + repr(res))
# 注：全部请求与响应写入 trace，作为复现与审计证据。
        self.trace.append(dict(kind=kind, x=p[0], y=p[1], channel=ch, response=res,
            virtual_time_s=self.client.virtual_time))
        if kind == 'measure':
            self.measure_count += 1
        else:
            self.clear_count += 1
            if res.get('clear_result') == 'success':
                self.cleared.add(ch)
        return res

```

文件：问题三/code/baseline_breakthrough.py

选读说明：问题三基础层：100 m 覆盖账本、任务路由、观测缓存与主循环（选摘）。

■ 100 m 单元覆盖账本

注：完成覆盖=每个 100 m 单元四角都落入某个已执行扫描的 1000 m 接收盘（充分条件，保守）。

CELL = 100.0

```

class Coverage:
    def __init__(self):
        self.cells = []
        for i in range(-18, 18):
            for j in range(-18, 18):
                x, y = (i+.5)*CELL, (j+.5)*CELL
                if math.hypot(max(0, abs(x)-CELL/2), max(0, abs(y)-CELL/2)) <= 1800:
                    self.cells.append((x, y))
        self.initial = len(self.cells)
        self.scans = []

    @staticmethod
# 注：covers(p,c): 扫描点 p 的接收盘是否覆盖单元 c 的四角。
    def covers(p, c):
        return (abs(p[0]-c[0])+CELL/2)**2 + (abs(p[1]-c[1])+CELL/2)**2 <= (1000-1e-5)**2

    def gain(self, p):
        return sum(self.covers(p, c) for c in self.cells)

    def update(self, p):

```

```

        self.cells = [c for c in self.cells if not self.covers(p, c)]
        self.scans.append(p)

# 注：候选扫描点=剩余单元抽样+两圈环形候选；按“新增覆盖/代价”贪心选择。
def next_point(self, pos):
    candidates = list(self.cells[::4])
    candidates += [mul(unit(a), r) for r in (1150, 1350) for a in range(0, 360, 30)]
    return max(candidates, key=lambda p: self.gain(p)/(dist(pos, p)+400)) if candidates else None

```

■ 任务集合与开放路径

```

def next_job(self):
    jobs = [('target', ch, enclosing_circle(tr['poly']))[0] for ch, tr in self.tracks.items()]
    jobs += [('scan', i, p) for i, p in enumerate(self.fixed_nodes)]
    if not jobs:
        return None
    start = tuple(self.client.position)
    # Cheapest insertion followed by open-path 2-opt. There is no return-to-
    # origin requirement; the final edge is intentionally absent.

```

注：任务集=已发现目标中心+6个固定外圈扫描点；开放路径（不返回起点）最近邻+2-opt。

```

    remaining = list(range(len(jobs)))
    route = []
    pos = start
    while remaining:
        k = min(remaining, key=lambda i: dist(pos, jobs[i][2]))
        remaining.remove(k); route.append(k); pos = jobs[k][2]
    for _ in range(8):
        improved = False
        for i in range(len(route)-1):
            before = start if i == 0 else jobs[route[i-1]][2]
            a = jobs[route[i]][2]
            for j in range(i+1, len(route)):
                b = jobs[route[j]][2]
                after = jobs[route[j+1]][2] if j+1 < len(route) else None
                old = dist(before, a) + (dist(b, after) if after else 0)
                new = dist(before, b) + (dist(a, after) if after else 0)
                if new < old-1e-6:
                    route[i:j+1] = reversed(route[i:j+1]); improved = True
                    break
            if improved:
                break
        if not improved:
            break
    return jobs[route[0]]

```

■ 观测缓存与可行域更新

```

def observe(self, p, ch):

```

注：同一位置同一频道的测向结果缓存，避免重复代价。

```

    key = (ch, round(p[0], 6), round(p[1], 6))
    if key in self._cache:
        return self._cache[key]
    res = self.action('measure', p, ch)
    self._cache[key] = res
    kind = res['measure_result']

```

注：near: 5 m内就地处清；direction 进入可行域维护。

```

    if kind == 'near':
        hit = self.action('clear', p, ch)
        if hit['clear_result'] != 'success':
            raise RobotError('near clear failed')
        self.unknown.discard(ch)
        self.tracks.pop(ch, None)
    elif kind == 'direction':
        t = res['svd_deg']
        self.unknown.discard(ch)

```

注：首次 direction 建立先验可行域；再次 direction 连续裁剪。

```

        if ch not in self.tracks:
            self.tracks[ch] = dict(poly=prior(p, t, ALPHA), s0=p, t0=t,
                                     last=p, theta=t, observations=1)
        else:
            tr = self.tracks[ch]
            tr['poly'] = clip_wedge(tr['poly'], p, t, ALPHA)
            tr.update(last=p, theta=t, observations=tr['observations']+1)
        if not self.tracks[ch]['poly']:
            raise RobotError('Inconsistent bearing polygon')
    return res

■ 扫描与共享测向
    def scan(self, p):
# 注：扫描点对所有未知频道逐一检测（问题三为全向源，未响应即排除）。
        # Every channel still unknown afterwards was checked at every scan disk.
        for ch in sorted(self.unknown, key=lambda c: (c != self.client.channel, c)):
            self.observe(p, ch)
        self.coverage.update(p)

    def share(self, p, exclude=None):
        if not self.shared:
            return
        for ch in sorted(list(self.tracks), key=lambda c: (c != self.client.channel, c)):
            if ch == exclude or ch in self.cleared:
                continue
            tr = self.tracks[ch]
            c, r = enclosing_circle(tr['poly'])
            baseline = dist(p, tr['last'])
# 注：共享测向条件：可行集仍大、距离≤1150 m、与上次示向角度差>0.12 rad 或足够近。
            angle = abs(math.sin(math.radians(bearing(p, c)-bearing(tr['last'], c))))
            if r > CLEAR_SAFE and dist(p, c) <= 1150 and baseline >= 60 and (angle > .12 or dist(p, c) <
150):
                self.observe(p, ch)
                self.shared_count += 1

■ 主循环
    def run(self):
        status = 'complete_coverage'
        try:
# 注：先在原点执行一次全频道扫描（原点必测）。
            self.scan((0., 0.))
            while True:
                if len(self.cleared) >= 16:
                    status = 'complete_upper_bound'
                    break
                self.tracks = {ch: tr for ch, tr in self.tracks.items() if ch not in self.cleared}
# 注：路由模式主循环：目标与固定扫描点统一排序；清除数达到题面上限 16 即提前结束。
                if self.routing and not self.adaptive and (self.tracks or self.fixed_nodes):
                    kind, index, p = self.next_job()
                    if kind == 'target':
                        self.engage(index)
                    else:
                        self.scan(self.fixed_nodes.pop(index))
                        self.visited_fixed += 1
                        self.share(tuple(self.client.position))
                elif self.tracks:
                    p = tuple(self.client.position)
                    ch = min(self.tracks, key=lambda c: dist(p,
enclosing_circle(self.tracks[c]['poly'])[0]))
                    self.engage(ch)
                elif self.adaptive and self.coverage.cells and self.unknown:
                    self.scan(self.coverage.next_point(self.client.position))
                    self.share(tuple(self.client.position))
                elif not self.adaptive and self.fixed_nodes:

```

```

        self.scan(self.fixed_nodes.pop(0))
        self.visited_fixed += 1
        self.share(tuple(self.client.position))
    else:
        break
except BudgetStop as exc:
    status = str(exc)
    return self.summary(status)

```

文件：问题三/code/prototype_v1.py

选读说明：问题三联合规划层：负观测裁剪、开放路径与覆盖规划（选摘）。

■ 负观测利用：圆外区域裁剪

注：把负观测转成“源在半径 999.9999 m 圆外”，与正观测共同裁剪可行域。

```

def exclude_disk(poly, p, radius=999.9999):
    # Convex hull of P outside disk. Extrema occur at polygon vertices or at
    # intersections of polygon edges with circle. Convexification is an outer
    # 注：对每条多边形边：保留圆外顶点，并求边与圆的交点一并加入。
    # relaxation, so even disconnected feasible pieces are never discarded.
    pts=[]
    for a,b in zip(poly,poly[1:]+poly[:1]):
        # 注：凸包是外松弛：即使可行域不连通也不会误删真实位置。
        if dist(a,p)>=radius: pts.append(a)
        v=sub(b,a); w=sub(a,p); aa=dot(v,v)
        if aa<1e-18: continue
        bb=2*dot(w,v); cc=dot(w,w)-radius*radius; disc=bb*bb-4*aa*cc
        if disc>=0:
            for t in ((-bb-math.sqrt(disc))/(2*aa),(-bb+math.sqrt(disc))/(2*aa)):
                if 0<=t<=1: pts.append(add(a,mul(v,t)))
    return hull(pts)

```

■ 开放路径规划

注：最近邻构造+2-opt 反转改进；问题三不要求返回起点。

```

def route_order(start,jobs):
    todo=list(jobs); out=[]; p=start
    while todo:
        k=min(range(len(todo)),key=lambda i:dist(p,todo[i][2])); j=todo.pop(k);out.append(j);p=j[2]
    for _ in range(80):
        improve=False
        for i in range(len(out)-1):
            a=start if i==0 else out[i-1][2]; b=out[i][2]
            for j in range(i+1,len(out)):
                c=out[j][2]; d=out[j+1][2] if j+1<len(out) else None
                if dist(a,c)+(dist(b,d) if d else 0)<dist(a,b)+(dist(c,d) if d else 0)-1e-6:
                    out[i:j+1]=reversed(out[i:j+1]);improve=True;break
            if improve:break
        if not improve:break
    return out

```

■ 联合覆盖规划（采样掩码）

```

def next_job(self):
    if not self.joint:return super().next_job()
    start=tuple(self.client.position)
    jobs=[('target',ch,enclosing_circle(tr['poly']))[0] for ch,tr in self.tracks.items()]
    future=self.covered_mask
    for j in jobs:future |= mask(j[2])
    full=(1<<len(SAMPLES))-1
    route=route_order(start,jobs)
    # 注：以离散采样掩码度量已计划覆盖的范围。
    while future != full:
        best=None
        for p,m in zip(self.candidates,self.candidate_masks):
            gain=(m&~future).bit_count()
            if not gain:continue
        # 注：每轮选择“（插入路程/新增覆盖）”最小的候选扫描点插入路径。
        for k in range(len(route)+1):

```

```

        a=start if k==0 else route[k-1][2];b=route[k][2] if k<len(route) else None
        cost=dist(a,p)+(dist(p,b)-dist(a,b) if b else 0)+60
        score=cost/gain
        if best is None or score<best[0]:best=(score,p,m,k)
        if best is None:raise ArithmeticError('coverage planning candidates insufficient')
        _,p,m,k=best;route.insert(k,('scan',None,p));future |= m
    route=route_order(start,route)
    if route:return route[0]
# 注：计划完成后用连续 Voronoi 证书复核；未通过则以证书见证点补扫。
    ok,w,d=certificate(self.coverage.scans)

self.coverage_proof=dict(complete=ok,worst_distance_m=d,witness=w,scans=list(self.coverage.scans))
    if ok:return None
    # Repair continuous gaps missed by planning samples, with an actual
    # scan centred on the witness. No sampled certificate can terminate.
    return ('scan',None,w)

```

■ 主循环

```

def run(self):
    if not self.joint:return super().run()
    from robot_core import BudgetStop
    status='complete_coverage'
    try:
# 注：先在原点扫描，再进入“定位-清除-覆盖”主循环。
        self.scan((0.,0.))
        for _ in range(200):
            if len(self.cleared)>=16:status='complete_upper_bound';break
            self.tracks={c:t for c,t in self.tracks.items() if c not in self.cleared}
            job=self.next_job()
# 注：证书通过（无可执行任务）才允许结束。
            if job is None:break
            if job[0]=='target':self.engage(job[1])
            else:self.scan(job[2]);self.share(tuple(self.client.position))
            else:status='iteration_limit'
        except BudgetStop as exc:status=str(exc)
    return self.summary(status)

```

文件：问题三/code/prototype_v6.py

选读说明：问题三巡回层：弹性六点巡回与圆周覆盖证书（全文）。

```

"""Elastic six-scan tour with exact angular cover preserved at every update.
For centres at radius <=1660, coverage of the 1800m boundary plus an origin
scan implies coverage of the whole disk (convex radial squared-distance).
"""

```

```

import math
from prototype_v5 import Strategy as V5
from prototype_v1 import route_order
from geometry import dist, mul, unit, enclosing_circle, add, sub, dot
from baseline_breakthrough import Strategy as Base

```

注：boundary_cover: 每个扫描点求其 999.99 m 接收盘与 1800 m 边界圆的可见弧段；弧段并集覆盖整周即边界被覆盖。

```

def boundary_cover(sites):
    arcs=[]
    for p in sites:
# 注：结合“径向平方距离关于 r 的凸性”：边界被覆盖即可推出整个圆盘被覆盖。
        r=math.hypot(*p)
        if r<800.1 or r>1660.0001:continue
        h=math.acos(max(-1,min(1,(1800**2+r*r-999.99**2)/(3600*r))))
        a=math.atan2(p[1],p[0])%(2*math.pi)
        lo,hi=a-h,a+h
        if lo<0:arcs.extend([(0,hi),(lo+2*math.pi,2*math.pi)])
        elif hi>2*math.pi:arcs.extend([(lo,2*math.pi),(0,hi-2*math.pi)])
        else:arcs.append((lo,hi))
    end=0.
    for lo,hi in sorted(arcs):

```

```

        if lo>end+1e-12:return False
        end=max(end,hi)
    return end>=2*math.pi-1e-12

class Strategy(V5):
    def __init__(self,client,mode='q3',*,elastic=True,passes=3,attempt=150,**kwargs):
        super().__init__(client,mode,replace=False,attempt=attempt,**kwargs)
        self.elastic=elastic;self.passes=passes;self.node_moves=0

# 注：任务=目标估计中心+固定扫描节点。
    def next_job(self):
        start=tuple(self.client.position)
        jobs=[('target',ch,enclosing_circle(tr['poly']))[0]] for ch,tr in self.tracks.items()
        route=route_order(start,jobs+[('scan',i,p) for i,p in enumerate(self.fixed_nodes)])
        if self.elastic:
            for _ in range(self.passes):
                changed=False
                for k,j in enumerate(route):
                    if j[0]!='scan':continue
                    i=j[1];p=self.fixed_nodes[i]
# 注：弹性更新：扫描节点可沿相邻任务连线滑移或小步移动，只接受仍保持边界覆盖的移动。
                    a=start if k==0 else route[k-1][2];b=route[k+1][2] if k+1<len(route) else None
                    if b is None:target=a
                    else:
                        v=sub(b,a);t=max(0,min(1,dot(sub(p,a),v)/max(1e-10,dot(v,v))))
                        target=add(a,mul(v,t))
                    candidates=[add(p,mul(sub(target,p),f)) for f in (1,.75,.5,.25,.125)]
                    candidates += [add(p,mul(unit(ang),step)) for step in (100,30) for ang in
range(0,360,45)]

                    old=dist(a,p)+(dist(p,b) if b else 0);best=old,p
                    for q in candidates:
                        r=math.hypot(*q)
                        if not 801<=r<=1660:continue
                        cost=dist(a,q)+(dist(q,b) if b else 0)
                        if cost>=best[0]-1e-5:continue
                        sites=self.coverage.scans+self.fixed_nodes[:i]+[q]+self.fixed_nodes[i+1:]
                        if boundary_cover(sites):best=cost,q
                        if best[0]<old-1e-5:

self.fixed_nodes[i]=best[1];route[k]=('scan',i,best[1]);changed=True;self.node_moves+=1
                    if not changed:break
                    route=route_order(start,jobs+[('scan',i,p) for i,p in enumerate(self.fixed_nodes)])
                return route[0]

    def summary(self,status):

out=super().summary(status);out.update(algorithm='ultimate200_v6',elastic_node_updates=self.node_move
s,elastic=self.elastic)
        out['coverage_certificate']='circular_arc_radial' if boundary_cover(self.coverage.scans) else
'not_yet_complete'
        return out

# 注：完成时再次验证圆周覆盖不变量（证明而非估计）。
    def run(self):
        # Use exactly the continuous disk contract preserved by next_job,
        # rather than require the strictly larger outer tangent polygon.
        result=Base.run(self)
        if result['status']=='complete_coverage' and not boundary_cover(self.coverage.scans):
            raise ArithmeticError('circular arc coverage invariant violated')
        return result

```

文件：问题三/code/prototype_v10.py

选读说明：问题三先导层：共享先导测向与提前终止（全文）。

```

"""Combine shared initial bearings with a continuously certified elastic tour."""
from prototype_v6 import Strategy as V6
from prototype_v7 import Strategy as V7

class Strategy(V6):
    def __init__(self,client,mode='q3',*,pilot=200,attempt=50,**kwargs):
        super().__init__(client,mode,attempt=attempt,**kwargs)
        self.pilot=pilot;self.pilot_done=False

# 注：首次到达原点后，用已发现目标的可行集中心选出信息收益最大的共享先导点。
    def scan(self,p):
        # V7.scan's super() needs V7 in the MRO, so share the small operation
        # through a helper proxy over exactly this public policy state.
        super().scan(p)
        if self.pilot_done or p!=(0.,0.):return
        self.pilot_done=True
        if len(self.tracks)<2 or not self.pilot:return
        import math
        from geometry import dist,unit,mul,enclosing_circle,bearing
        estimates=[enclosing_circle(t['poly']) for t in self.tracks.values()]
        best=None
        for a in range(0,360,15):
            q=mul(unit(a),self.pilot);loss=0.
            for c,r in estimates:
# 注：损失代理：对每个目标取 min(可行集半径, 0.035*距离/sin 交会角) 之和，越小越好。
                s=abs(math.sin(math.radians(bearing(q,c)-bearing(p,c))))
                loss+=min(r,.035*dist(q,c)/max(s,.015))
            if best is None or loss<best[0]:best=loss,q
        for ch in sorted(list(self.tracks),key=lambda c:(c!=self.client.channel,c)):
            self.observe(best[1],ch)

# 注：已清除≥16 个频道后停止共享测向（题面上限）。
    def share(self,p,exclude=None):
        if len(self.cleared)<16:super().share(p,exclude)

    def summary(self,status):
        out=super().summary(status);out.update(algorithm='ultimate200_v10',pilot_m=self.pilot);return
out

```

文件：问题三/code/strategy.py

选读说明：问题三最终冻结配置（全文）。

```

"""Frozen combo100 policy. Public protocol only; standard-library runtime."""
# 注：最终策略=v10 基类+100 m 共享先导+50 m 试清半径+共享开启（combo100）。
from prototype_v10 import Strategy as Combo

```

```

class Strategy(Combo):
    def __init__(self,client,mode='q3',*,shared=True):
        super().__init__(client,mode,pilot=100,attempt=50,shared=shared)

    def summary(self,status):
        out=super().summary(status)
        out['algorithm']='ultimate200_frozen_combo100'
        return out

```

文件：问题三/code/coverage_exact.py

选读说明：问题三连续覆盖证书：裁剪 Voronoi 单元（全文）。

```

"""Conservative continuous coverage certificate via clipped Voronoi cells.
Outer regular polygon contains the entire radius-1800 disk. A convex squared
distance reaches its maximum at a vertex of each clipped Voronoi cell.
Sampling is used for planning only, never for completion certification.
"""
import math
from geometry import unit, mul, clip, dist, dot

```

```

# 注：72 边外切正多边形覆盖整个 1800 m 圆盘（只放大域，不漏检）。
OUTER = [mul(unit((i+.5)*5), 1800/math.cos(math.radians(2.5))) for i in range(72)]
SAMPLES = [mul(unit(a), r) for r, step in [(600,30),(1100,15),(1500,10),(1800,5)] for a in range(0,360,step)]
+ [(0.,0.)]

# 注：连续覆盖证书：每个站点裁出相对其他站点的 Voronoi 单元，凸平方距离在单元顶点取最大。
def certificate(sites):
    if not sites: return False, (1800.,0.), float('inf')
    worst = -1.; witness = None
    for i,s in enumerate(sites):
        poly = list(OUTER)
        for j,t in enumerate(sites):
            # 注：最远 Voronoi 顶点<1000 m ⇒ 圆盘内任一点都在某站点接收半径内。
            if i == j or dist(s,t)<1e-8: continue
            poly = clip(poly, (2*(t[0]-s[0]), 2*(t[1]-s[1]), dot(t,t)-dot(s,s)))
            if not poly: break
        for v in poly:
            d = dist(v,s)
            if d>worst: worst,witness=d,v
    return worst < 1000-1e-4, witness, worst

# 注：采样掩码只用于规划启发，不用于完成判定。
def mask(p):
    return sum(1<<i for i,q in enumerate(SAMPLES) if dist(p,q)<990)

```

B.3 问题四：可见性覆盖与持久定位清除

问题四把“任意发射方向可见”化为局部三角片上的凸包包含条件，以三角网节点为扫描布局；目标定位沿用可行集内核，末端用矩形光学回退保证清除。

文件：问题四/code/run_q4.py

选读说明：问题四命令入口（全文）。

```

"""Run only when the official Q4 simulator interface is ready; never starts a case."""
import argparse,json,time,uuid
from pathlib import Path
from robot_client import RobotClient,RobotError
from strategy_selected import Strategy

def main():
    p=argparse.ArgumentParser(description=__doc__);p.add_argument('--robot-id',required=True)

    p.add_argument('--url',default='http://127.0.0.1:2026');p.add_argument('--out',default='windows_results_q4')

    a=p.parse_args();folder=Path(a.out)/(time.strftime('%Y%m%d_%H%M%S')+ '_q4_'+uuid.uuid4().hex[:6]);folder.mkdir(parents=True)
    c=RobotClient(a.url,a.robot_id,log_path=str(folder/'client.jsonl'));st=None;t0=time.perf_counter()
    try:
        # 注：进入、运行、退出；异常路径不在未知网络结果后追加动作。
        status,res=c.enter()
        if status!=200 or res.get('accepted') is not True:raise RobotError('enter rejected: '+str(res))
        st=Strategy(c,'q4');summary=st.run()
        if c.time_left() is not None and c.time_left()>2:
            status,res=c.exit()
            if status!=200 or res.get('accepted') is not True:raise RobotError('exit rejected')
    except Exception as exc:
        summary=st.summary('error') if st is not None else dict(mode='q4',status='error',cleared=0)
        summary['error']=repr(exc)
    # 注：记录含初始化的墙钟时间（与正式表口径不同，表 17 用 /enter→/exit 响应时间戳）。
    summary['program_wall_s_including_init']=time.perf_counter()-t0
    summary.update(official_case_code=None,official_total_sources=None,official_encrypted_log=None)

    (folder/'summary.jsonl').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')

```

```

    if st is not None:
        (folder/'trace.json').write_text(json.dumps(st.trace,indent=2),encoding='utf-8')
        (folder/'certificates.json').write_text(json.dumps(st.certificates,indent=2),encoding='utf-8')

c.close_log(summary['status']);print(json.dumps(summary,ensure_ascii=False,indent=2));print(folder.resolve())
    return 0 if summary['status'].startswith('complete') else 2
if __name__=='__main__':raise SystemExit(main())
文件：问题四/code/coverage.py
选读说明：问题四扫描布局：三角网单元与 22 点环（全文）。
"""Continuous Q4 certificate: each intersecting cell's vertices surround its sources.
Every cell diameter is <1000 m. At least one vertex lies in every closed
directional half-plane through any point in the cell. No orientation sampling.
"""

import math
from geometry import dist, dot, sub, cross

def origin_distance(poly):
    signs=[cross(sub(b,a),(-a[0],-a[1])) for a,b in zip(poly,poly[1:]+poly[:1])]
    if all(v>=-1e-9 for v in signs) or all(v<=1e-9 for v in signs):return 0.
    result=math.inf
    for a,b in zip(poly,poly[1:]+poly[:1]):
        v=sub(b,a);t=max(0.,min(1.,-dot(a,v)/dot(v,v)))
        result=min(result,math.hypot(a[0]+t*v[0],a[1]+t*v[1]))
    return result

# 注：两种单元构造：默认三角网（间距 990 m）与方格网。
def mesh(kind='triangle',spacing=990.,phase=0.):
    if kind=='square':
        h=spacing/math.sqrt(2);n=math.ceil(1800/h)+2
        cells=[[ (i*h,j*h),((i+1)*h,j*h),((i+1)*h,(j+1)*h),(i*h,(j+1)*h)]
                for i in range(-n,n) for j in range(-n,n)]
    else:
        h=spacing*math.sqrt(3)/2;n=math.ceil(1800/h)+3
        def p(i,j):return (spacing*(i+j*.5),h*j)
        cells=[]
        for i in range(-n,n):
            for j in range(-n,n):
                cells.extend([[p(i,j),p(i+1,j),p(i,j+1)], [p(i+1,j),p(i+1,j+1),p(i,j+1)]])
# 注：只保留与 1800 m 目标圆盘相交的单元。
    cells=[c for c in cells if origin_distance(c)<=1800+1e-7]
    a=math.radians(phase);co,si=math.cos(a),math.sin(a)
    cells=[[ (co*x-si*y,si*x+co*y) for x,y in c] for c in cells]
    nodes=sorted(set(v for c in cells for v in c))
# 注：断言单元直径<1000 m，保证单元内任意点到顶点距离上界（供覆盖证书使用）。
    assert max(dist(a,b) for c in cells for a in c for b in c)<1000
    return nodes,cells
文件：问题四/code/coverage_certificate.py
选读说明：问题四连续可见性证书：自适应三角片+凸包包含（全文）。
"""Sufficient continuous certificate for arbitrary Q4 scan sites.
Adaptive triangles tile an outer regular 96-gon containing the true disk.
For each tile C, use only sites within 999.99m of every vertex of C.
If C is inside their convex hull, every source in C sees at least one site
for every closed directional half-plane. Subdivision reduces conservatism.
Failure means unproved, never a permission to stop scanning.
"""

import math
from geometry import dist,hull,cross,sub,unit,mul,add

def contains(poly,p,tol=1e-7):
    return len(poly)>=3 and all(cross(sub(b,a),sub(p,a))>=-tol for a,b in zip(poly,poly[1:]+poly[:1]))

```

```

# 注：96 边外切多边形细分为三角形（初始 96 片），做连续覆盖证明。
def certify(nodes,max_depth=18,max_tiles=100000):
    radius=1800/math.cos(math.pi/96)
    boundary=[mul(unit(i*360/96),radius) for i in range(96)]
    stack=[((0.,0.),boundary[i],boundary[(i+1)%96]),0) for i in range(96)]
    passed=0;checked=0;depth_seen=0
    while stack:
        tri,depth=stack.pop();checked+=1;depth_seen=max(depth_seen,depth)
# 注：局部站点集：到三角形三个顶点均≤999.99 m 的扫描点。
        local=[p for p in nodes if all(dist(p,v)<=999.99 for v in tri)]
        poly=hull(local)
# 注：三顶点被局部站点凸包包含 ⇒ 片内任意位置、任意发射方向至少一个站点在闭半平面内可见。
        if all(contains(poly,v) for v in tri):passed+=1;continue
        if depth>=max_depth or checked>=max_tiles:
            return
    dict(certified=False,checked=checked,passed=passed,max_depth=depth_seen,unproved_triangle=tri)
# 注：未通过则二分最长边继续；达到深度/规模上限返回“未证明”（保守失败，绝不意味着可停止扫描）。
    i,j=max(((0,1),(1,2),(2,0)),key=lambda ij:dist(tri[ij[0]],tri[ij[1]]))
    k=3-i-j;m=mul(add(tri[i],tri[j]),.5)
    stack.extend([(tri[i],m,tri[k]),depth+1],[m,tri[j],tri[k]),depth+1])
    return
dict(certified=True,checked=checked,passed=passed,max_depth=depth_seen,domain_outer_sides=96,distance_limit_m=999.99)

if __name__=='__main__':
    import argparse,json
    from pathlib import Path
    p=argparse.ArgumentParser();p.add_argument('layout');a=p.parse_args()
    data=json.loads(Path(a.layout).read_text());r=certify(data['nodes']);print(json.dumps(r))
    raise SystemExit(0 if r['certified'] else 1)

```

文件：问题四/code/strategy.py

选读说明：问题四最终策略：持久定位、共享测向、矩形光学回退（选摘）。

■ 冻结参数与布局

```

class Strategy(StrategyBase):
# 注：冻结参数：三角网间距 990 m、相位 0、路由 twoopt、局部器 recover、共享测向开、试清半径 60 m、批量扫描。
    def __init__(self,client,mode='q4',*,routing='twoopt',mesh_kind='triangle',spacing=990.,
        phase=0.,localizer='recover',shared=True,trial_radius=60.,batch=True):
        if mode!='q4':raise ValueError('Q4 only')
        super().__init__(client,mode)
        self.unknown=set(range(1,21));self.tracks={};self.cache={}
# 注：nodes 即 coverage.mesh 产生的 22 点布局（与图 18、表 14 一致）。
        self.nodes,self.cells=mesh(mesh_kind,spacing,phase);self.all_nodes=list(self.nodes)
        self.scans=[];self.certificates=[];self.routing=routing;self.localizer=localizer
        self.shared=shared;self.trial_radius=trial_radius;self.batch=batch
        self.shared_count=0;self.recoveries=0;self.route_calls=0

```

■ 观测缓存、单点扫描与共享测向

```

    def observe(self,p,ch):
# 注：同一位置同一频道的测量缓存。
        key=(ch,round(p[0],6),round(p[1],6))
        if key in self.cache:return self.cache[key]
        res=self.action('measure',p,ch);self.cache[key]=res;k=res['measure_result']
# 注：near: 5 m 内就地处清。
        if k=='near':
            if self.action('clear',p,ch)['clear_result']!='success':raise RobotError('near failed')
            self.unknown.discard(ch);self.tracks.pop(ch,None)
# 注：新目标以首次示向楔形建立先验可行域。
        elif k=='direction':
            self.unknown.discard(ch)
            if ch not in self.tracks:
                self.tracks[ch]=dict(poly=prior(p,res['svd_deg'],ALPHA),s0=p,t0=res['svd_deg'],
                    last=p,theta=res['svd_deg'],obs=1,tried=[],optical=False)
            else:

```

```

        tr=self.tracks[ch];tr['poly']=clip_wedge(tr['poly'],p,res['svd_deg'],ALPHA)
        tr.update(last=p,theta=res['svd_deg'],obs=tr['obs']+1)
        if not self.tracks[ch]['poly']:raise RobotError('Empty feasible region')
    return res
def scan(self,p):
    # Batch measures before any localization; near clears occur at p itself.
# 注: 批量扫描: 一个扫描点检测全部未知频道; 清满 16 个频道立即返回。
    for ch in sorted(list(self.unknown),key=lambda c:(c!=self.client.channel,c)):
        self.observe(p,ch)
        if len(self.cleared)==16:return
    self.scans.append(p)
    self.share(p)
def share(self,p,exclude=None):
    if not self.shared or len(self.cleared)==16:return
    for ch in sorted(list(self.tracks),key=lambda c:(c!=self.client.channel,c)):
        if ch==exclude or ch in self.cleared:continue
        tr=self.tracks[ch];c,r=enclosing_circle(tr['poly'])
        angle=abs(math.sin(math.radians(bearing(p,c)-bearing(tr['last'],c))))
# 注: 共享测向条件: 可行集半径>19.9 m、距离<1200 m、与上次示向角度差>0.12 rad (或距离<150 m)。
        if r>CLEAR_SAFE and dist(p,c)<1200 and dist(p,tr['last'])>60 and (angle>.12 or dist(p,c)<150):
            self.observe(p,ch);self.shared_count+=1
            if len(self.cleared)==16:return
■ 认证清除与矩形光学回退
def clear_at(self,p,ch,certificate=None):
    ok=self.action('clear',p,ch)['clear_result']=='success'
    if certificate is not None:
        if not ok:raise RobotError('Certified clear failed')
        self.certificates.append(certificate)
    if ok:self.tracks.pop(ch,None)
    return ok
def sweep(self,ch):
# 注: 矩形回退: 在直径坐标系内以≤28×28 m 网格覆盖剩余可行域。
    """Tight rectangle covering the *remaining* polygon, optical distance only.
    Local grid ≤28 x28 m gives radius ≤sqrt(142+142)<20 m.
    Then nearest-neighbour/2opt visits every point unless success occurs.
    """
    self.fallback_count+=1;tr=self.tracks[ch];poly=tr['poly']
    info=diameter_info(poly);a,b=info['diameter_pair'];t=bearing(a,b);u=unit(t);v=(-u[1],u[0])
    coords=[(dot(sub(p,a),u),dot(sub(p,a),v)) for p in poly]
    xs=[p[0] for p in coords];ys=[p[1] for p in coords]
    lo,hi=min(xs),max(xs);bot,top=min(ys),max(ys)
# 注: 网格数: 每个单元中心到单元内任意点≤√(142+142)≈19.80 m<20 m。
    nx=max(1,math.ceil((hi-lo)/28));ny=max(1,math.ceil((top-bot)/28))
    pts=[local_to_global(a,t,(lo+(i+.5)*(hi-lo)/nx,bot+(j+.5)*(top-bot)/ny)) for i in range(nx) for
j in range(ny)]
# 注: 按最近邻+2-opt 访问全部单元中心, 直至成功。
    order=route(self.client.position,pts,'twoopt')
    for i in order:
        if self.clear_at(pts[i],ch):return
    raise RobotError('Complete feasible rectangle sweep failed')
■ 目标清除流程
def engage(self,ch):
    if ch not in self.tracks:return
    tr=self.tracks[ch]
    for attempt in range(8):
        if ch in self.cleared:return
# 注: r≤19.9 m: 认证清除并记录证书。
        tr=self.tracks[ch];c,r=enclosing_circle(tr['poly'])
        if r<=CLEAR_SAFE:
            self.clear_at(c,ch,dict(channel=ch,center=c,radius=r,vertices=tr['poly']))
            self.share(self.client.position);return
# 注: r≤60 m: 允许一次试清 (仅成功计入清除)。
```

```

        if r<=self.trial_radius and not tr['optical']:
            tr['optical']=True
            if self.clear_at(c,ch):self.share(self.client.position);return
# 注: 已至少两次示向或已试两轮 ⇒ 转入光学回退。
        if self.localizer=='optical' and (tr['obs']>=2 or attempt>=2):break
        v=unit(tr['theta']+90);offset=max(25,min(160,.25*r))
        if tr['obs']==1:
            center=add(tr['last'],mul(unit(tr['theta']),300))
            candidates=[add(center,mul(v,75)),add(center,mul(v,-75))]
            # Remaining candidates reduce forward overshoot or gather broad parallax.
            candidates += [add(tr['last'],mul(v,sgn*80)) for sgn in (1,-1)]
        else:
            candidates=[add(c,mul(v,offset)),add(c,mul(v,-offset)),
                        mul(add(c,tr['last']),.5)]
            candidates += [add(tr['s0'],mul(sub(c,tr['s0']),f)) for f in (.25,.5,.75)]
            candidates=[p for p in candidates if all(dist(p,q)>1 for q in tr['tried'])
                        and (ch,round(p[0],6),round(p[1],6)) not in self.cache]
            if not candidates:break
            p=min(candidates,key=lambda p:dist(self.client.position,p)) if attempt else candidates[0]
# 注: 每次观测后共享本点对其他目标的测向机会。
            tr['tried'].append(p);res=self.observe(p,ch);self.share(self.client.position,exclude=ch)
            if res['measure_result']=='no_signal':
                if self.localizer=='strip':break
                self.recoveries+=1
            if ch not in self.cleared:self.sweep(ch);self.share(self.client.position)

```

■ 主循环与统计

注: 主循环: 扫描-路由-定位/清除; 清满 16 或节点与目标处理完毕。

```

def run(self):
    status='complete_coverage'
    try:
        self.scan((0.,0.));self.nodes=[p for p in self.nodes if dist(p,(0,0))>1e-7]
        while len(self.cleared)<16:
            job=self.next_job()
            if job is None:break
            kind,i,p=job
            if kind=='scan':self.nodes.pop(i);self.scan(p)
            else:self.engage(i)
# 注: 清满题面上限 16 ⇒ complete_upper_bound。
        if len(self.cleared)==16:status='complete_upper_bound'
        elif self.nodes or self.tracks:raise RobotError('Incomplete coverage')
        except BudgetStop as exc:status=str(exc)
        return self.summary(status)
    def summary(self,status):
        out=super().summary(status)

out.update(algorithm='q4_persistent',full_scan_count=len(self.scans),pending_nodes=len(self.nodes),
pending_tracks=len(self.tracks),shared_measures=self.shared_count,recoveries=self.recoveries,
mesh_nodes=len(self.all_nodes),route_calls=self.route_calls)
return out

```

文件: 问题四/code/routing.py

选读说明: 开放路径规划组合 (全文)。

"""Open Euclidean route portfolio; exact DP only certifies the current point set."""

import math, random

from functools import lru_cache

from geometry import dist

def length(start, points, order):

return sum(dist(start if k == 0 else points[order[k-1]], points[i]) for k,i in enumerate(order))

def improve(start, points, route):

route=list(route)

```

for _ in range(60):
    best=0.; move=None
    for i in range(len(route)-1):
        prev=start if i==0 else points[route[i-1]]; a=points[route[i]]
        for j in range(i+1,len(route)):
            b=points[route[j]]; nxt=points[route[j+1]] if j+1<len(route) else None
            gain=dist(prev,a)-dist(prev,b)
            if nxt is not None:gain+=dist(b,nxt)-dist(a,nxt)
            if gain>best+1e-7:best=gain;move=(i,j)
    if move is None:break
    i,j=move;route[i:j+1]=reversed(route[i:j+1])
return route

# 注：路径规划组合：最近邻、2-opt、多起点、退火；≤13 点用精确 DP。
def route(start,points,method='twoopt',seed=0):
    n=len(points)
    if not n:return []
    ds=[[dist(a,b) for b in points] for a in points]
# 注：精确 DP（小实例）给出可对照的最短开放路径。
    if method=='exact' and n<=13:
        @lru_cache(None)
        def dp(last,mask):
            if not mask:return (0.,-1)
            choices=[(ds[last][j]+dp(j,mask^(1<<j))[0],j) for j in range(n) if mask>>j&1]
            return min(choices)
        mask=(1<<n)-1
        _,first=min((dist(start,points[j])+dp(j,mask^(1<<j))[0],j) for j in range(n))
        out=[first];mask^=1<<first
        while mask:
            first=dp(first,mask)[1];out.append(first);mask^=1<<first
        return out
    remaining=set(range(n));order=[];pos=start
    while remaining:
        j=min(remaining,key=lambda
k:(dist(pos,points[k]),k));order.append(j);remaining.remove(j);pos=points[j]
        if method=='nearest':return order
# 注：多起点：旋转与随机重启后 2-opt，取更短者。
    best=improve(start,points,order)
    if method in ('multistart','anneal','exact'):
        rng=random.Random(seed)
        for trial in range(6):
            order=list(best)
            if trial<3:
                order=order[trial+1:]+order[:trial+1]
            else:rng.shuffle(order)
            order=improve(start,points,order)
            if length(start,points,order)<length(start,points,best):best=order
    if method=='anneal':
# 注：退火阶段接受临时劣化，末段再 2-opt 收尾。
        cur=list(best);val=length(start,points,cur);bv=length(start,points,best)
        for k in range(240):
            i,j=sorted(rng.sample(range(n),2)) if n>1 else (0,0)
            nxt=cur[:i]+list(reversed(cur[i:j+1]))+cur[j+1:]
            nv=length(start,points,nxt);temp=150*(1-k/240)+.1
            if nv<val or rng.random()<math.exp(min(0,(val-nv)/temp)):cur, val=nxt,nv
            if val<bv:best,bv=list(cur),val
        best=improve(start,points,best)
    return best
文件：问题四/code/strategy_selected.py
选读说明：问题四冻结策略略加载（全文）。
"""Frozen new candidate; previous deliverable remains independent."""
import json

```

```

from pathlib import Path
from strategy_explore import Strategy as Explore
from strategy_ring21 import Strategy as Ring21
from strategy_particle import Strategy as Particle
from strategy_active import Strategy as Active
from strategy_final import Strategy as Previous
CFG=json.loads((Path(__file__).resolve().parent/'selected_new.json').read_text(encoding='utf-8'))
# 注: NAME 决定基类: previous / active / particle / ring21 / explore (由 selected_new.json 冻结)。
NAME=CFG['selected']
BASE=Previous if NAME=='previous' else Active if NAME.startswith('active') else Particle if
NAME.startswith('particle') else Ring21 if NAME.endswith('21') else Explore
class Strategy(BASE):
    def __init__(self,client,mode='q4'):
        super().__init__(client,mode,**CFG['kwargs'])

```

B.4 公共接口

与模拟器的全部交互仅通过公开 HTTP 接口完成; 重试幂等、请求日志等完整实现见支撑材料中的同文件全文。

文件: 问题四/code/robot_client.py

选读说明: 模拟器公开接口客户端 (选摘动作方法)。

注: 进入测试: 记录虚拟时间、位置、频道与剩余现实时间, 并设置软截止。

```

def enter(self):
    payload = self._base_payload("enter")
    status, response = self._post("/enter", payload)
    if self._accepted(status, response):
        self.entered = True
        self.virtual_time = float(response.get("virtual_time_s", 0.0))
        self.position = (0.0, 0.0)
        self.channel = 1
        self.remaining_real_duration_s = float(response.get("remaining_real_duration_s", 0.0))
        self._deadline_monotonic = time.monotonic() + self.remaining_real_duration_s
    return status, response

```

注: measure: 提交位置与频道, 返回 direction/near/no_signal 与示向度。

```

def measure(self, x, y, channel):
    payload = self._base_payload("measure")
    payload["position"] = {"x": x, "y": y}
    payload["channel"] = channel
    status, response = self._post("/measure", payload)
    if self._accepted(status, response):
        self.virtual_time = float(response["virtual_time_s"])
        self.position = (float(x), float(y))
        self.channel = int(channel)
    return status, response

```

注: clear: 返回 success / no_target_in_range。

```

def clear(self, x, y, channel):
    payload = self._base_payload("clear")
    payload["position"] = {"x": x, "y": y}
    payload["channel"] = channel
    status, response = self._post("/clear", payload)
    if self._accepted(status, response):
        self.virtual_time = float(response["virtual_time_s"])
        self.position = (float(x), float(y))
    return status, response

```

注: exit: 结束测试并收敛虚拟时间。

```

def exit(self):
    payload = self._base_payload("exit")
    status, response = self._post("/exit", payload)
    if self._accepted(status, response):
        self.virtual_time = float(response.get("virtual_time_s", self.virtual_time))
    return status, response

```

以上为本论文核心算法的全部关键代码。完整源程序、运行配置、结果表与逐场证据（含六份正式加密日志）见支撑材料压缩包。